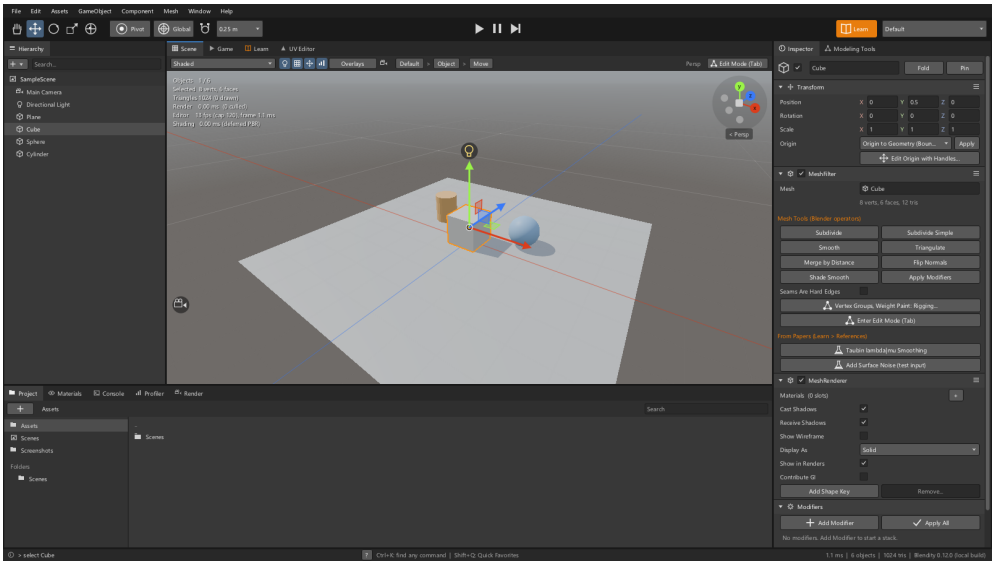
Blendity

The User Guide

Blender’s modeling, UVs, materials, rendering, rigging and animation, driven like Unity



Emiliano Manalo, with Anthropic’s Claude

Version 0.13.0 November 2026

Blendity is free software (GPL). Your scenes, meshes, textures and renders are yours.

https://github.com/EmilianoManaloIV/blendity

How to read this guide

Blendity is a 3D editor that works like Unity (windows, navigation, gizmos, the Inspector, Play mode) and models, unwraps, textures, renders, rigs and animates the way Blender does. This guide takes you through the editor in the order you will meet it: installing it, finding your way around, and then the work stage by stage. If you come from another program, chapter 3 is written for you: it puts your keys and your habits beside Blendity's and says what is where.

Three things to know before you start. **The Learn tab** (F1) teaches every part of the editor interactively, with Try-it buttons that drive the editor while you watch; this guide names the lesson to open at the end of each chapter. **The command palette** (Ctrl+K) finds any command by name, with its shortcut beside it. **Nothing is hidden for good**: a stage's layout shows that stage's tools first, and Window > Layouts > Show All Tools shows everything.

Conventions. Ctrl+K is a key or a chord; Window > Layouts is a path through the menus; Generate Lighting is a button, field or window as it is labelled; `select Cube` is a console command, a file or a path; a *Learn* box at the end of a chapter names the lessons to open next; a "Coming from" box says what a Unity or Blender user should know at that point; an "Example" box is a short task to follow step by step.

Every idea Blendity borrows is credited: the References chapter lists the papers, books, manuals and programs behind the features, with the pages where each is used, and Index C ties each Learn lesson to its sources.

Contents

How to read this guide **ii**

1 Installing Blendity and the first start **1**

1.1 Download . . . 1
1.2 Install . . . 1
1.3 The first start . . . 2
1.4 Starting from the command line . . . 2
1.5 Privacy . . . 2

2 Getting around **4**

2.1 The windows . . . 4
2.2 Layouts for each stage . . . 5
2.3 Navigating . . . 6
2.4 Selecting and moving . . . 6
2.5 Objects, components and the Inspector . . . 6
2.6 Origins, Play, undo . . . 7
2.7 Finding anything: the palette, favorites, the Wheel . . . 7
2.8 The status bar, toasts and the job tray . . . 7
2.9 Windows on other screens and Tablet Mode . . . 7
2.10 Themes . . . 8

3 Coming from another program **9**

3.1 The editor feel . . . 9
3.2 From Unity . . . 9
3.3 From Blender . . . 10
3.4 From Maya . . . 10
3.5 From 3ds Max . . . 11
3.6 From SketchUp . . . 11
3.7 From Unreal . . . 11
3.8 From Photoshop, Substance Painter and SketchBook . . . 11
3.9 The shortcut table . . . 12

4 Reference, planning and blocking out **13**

4.1 Reference images and the board . . . 13
4.2 Cameras and shots . . . 13
4.3 Blocking out . . . 13
4.4 Drawing in perspective, building from the construction . . . 14

5 Modeling, sculpting and UVs **16**
5.1 Edit Mode . . . 16
5.2 Sculpting . . . 17
5.3 UV unwrapping . . . 17

6 Materials, textures and painting **19**
6.1 Materials . . . 19
6.2 Procedural textures: Blender's nodes as layers . . . 19
6.3 Painting the surface: the Substance Painter pipeline . . . 21
6.4 Stylized environments and the anime look . . . 21
6.5 Bake Textures . . . 22

7 Lighting and rendering **23**
7.1 Lights, the world and the views . . . 23
7.2 F12 and the Render window . . . 23
7.3 Baked, mixed and realtime light . . . 24
7.4 Camera filters . . . 25
7.5 Developing a render: the Editing tab . . . 25
7.6 A scene opened from Unity . . . 25

8 Rigging, skinning and animation **27**
8.1 Skeletons and skins . . . 27
8.2 The Animation window . . . 28
8.3 Puppet Posing and physics for animation . . . 29
8.4 Retargeting . . . 29
8.5 Sequencing clips . . . 29

9 Files, other programs and Live Link **31**
9.1 The project . . . 31
9.2 Import . . . 31
9.3 Export . . . 31
9.4 Unity projects, VRChat, Godot, Blender, Unreal . . . 31
9.5 Live Link . . . 32

10 The Learn tab and the console **33**
10.1 The Learn tab . . . 33
10.2 The console . . . 34
10.3 Headless runs and scripts . . . 36
10.4 Checking how it works for you . . . 36

11 Pen, fingers, the Draw tab, papers, the board and VR **37**
11.1 Fingers and a pen on a 2-in-1 . . . 37
11.2 A pen display's dial and express keys . . . 37
11.3 The Draw tab . . . 37
11.4 Papers . . . 38
11.5 The board . . . 38
11.6 VR View . . . 39

12 Performance, the minimum machine and troubleshooting **40**
12.1 The minimum machine and the quality tiers . . . 40
12.2 When something refuses . . . 40
12.3 Without Blender's libraries . . . 41
12.4 The GPU . . . 41
12.5 Files and projects . . . 41
12.6 Where to ask . . . 41

13 Licensing and credit **42**
13.1 The terms . . . 42
13.2 Why these terms are compatible with Blender's . . . 42
13.3 Compatible with an AI-assisted project . . . 43
13.4 Due credit . . . 43

References **45**

Index C: the Learn lessons and their references **55**

Colophon **62**

List of Figures

2.1 The Default layout: Hierarchy, Scene view, Inspector and Modeling Tools, Project and Console; the toolbar's tools, pivot, space and snap increment; Play, Pause and Step; the Learn button and the Layout dropdown at the top right. 4

2.2 The Modeling & Topology layout in Edit Mode: the Modeling Tools window shows the Edit Mode sections (Select, Create & Extrude, Cut & Divide, Merge & Clean Up), the Draw tools and the checks. 6

5.1 The UV Unwrapping layout: Edit Mode on the cube, the UV Editor's toolbar (Unwrap, Smart UV, Part Unwrap, the projections, Pack, Mark Seam; vertex / face / island selection; the transforms) and the Modeling Tools beside it. 18

7.1 The Lighting & Rendering layout: the Scene view, the Game view through the Main Camera, the Lighting window (Generate Lighting, the lightmapping settings) and the Render window with its presets. 24

8.1 The Animation layout: the Animation window along the bottom with Preview, Record, the clip, Add Property, Puppet, Retarget, Dopesheet / Curves, Walk Cycle and the physics tools; the Rigging tab beside the Inspector. 28

10.1 The Learning layout: the lesson list, a lesson with its table, references and Try-it buttons, and the Scene view the buttons drive. 33

11.1 The Drawing layout: the Draw tab's pad with its toolbar, the Layers window and the Project window. 38

Chapter 1

Installing Blendity and the first start

Blendity is free. You don't need Blender or anything else: the installer brings everything Blendity uses. Version 0.13.0 is a **pre-release**: it works, but it is still being refined, so expect rough edges and save your work often.

1.1 Download

Open the Releases page, `https://github.com/EmilianoManaloIV/blendity/releases`, or click the link for your computer in the project's README: it fetches the one file you need, always the newest version.

Your computer	The file
Windows 10 or 11	`Blendity-Setup-Windows.exe`
A Mac with an Apple chip (M1 or later)	`Blendity-macos.dmg`
Linux (Ubuntu 22.04 or newer, Fedora, Mint, ...)	`Blendity-linux-x86_64.AppImage`

Not sure which Mac you have? Apple menu > About This Mac: "Chip: Apple M..." means an Apple chip. Macs with an Intel processor aren't supported yet. Every release also carries `SHA256SUMS.txt` (Blendity's updater checks downloads against it), this guide and the architecture book as PDFs, and "Source code" archives that are for developers.

1.2 Install

Windows. Double-click `Blendity-Setup-Windows.exe`. If a blue box says "Windows protected your PC", click More info, then Run anyway (Blendity isn't signed with a paid certificate yet). Pick for me only if asked (no administrator needed), accept the licence, keep Blendity with Blender's libraries (recommended), tick a desktop shortcut if you want one, Install, Finish. Next time, open it from the Start menu. To uninstall: Settings > Apps > Installed apps > Blendity > Uninstall; your projects are kept.

Mac. Double-click the `.dmg`, drag Blendity onto Applications, open it from Applications. The first time macOS says it can't check the app: on macOS 15 or newer click Done, then System Settings > Privacy & Security > Open Anyway; on macOS 14 or older right-click Blendity > Open > Open. If macOS says Blendity "is damaged", run `xattr -dr com.apple.quarantine /Applications/Blendity.app` in Terminal once. To uninstall, drag it to the Bin.

Linux. Right-click the `.AppImage` > Properties > Permissions > Allow executing file as program, then double-click it. If nothing happens, download the plain folder (`blendity-linux-x86_64.tar.gz` from the "libraries" release), extract it and double-click Blendity inside. To uninstall, delete the file or the folder.

1.3 The first start

- The welcome asks **where you come from** (Unity, Unreal, Blender, Photoshop & Substance) and sets the keys and the editor feel together; Help > Welcome asks again, and chapter 3 explains each choice.
- Your work is saved in a **project folder**: `BlendityProject` in your home folder unless you open another. A project is just a folder with `Assets` inside, like Unity's; copy it to another machine and open a scene in it. The program's folder belongs to the installer; your scenes, materials and renders never live there.
- Press F1 for the **Learn tab**: short interactive lessons for each part of Blendity, with Try-it buttons that drive the editor while you watch (chapter 10).
- **Updates:** once a day, when online, Blendity asks GitHub whether a newer build exists and shows what it brings with Install / Not now / Skip this one; installs are checked against the release's SHA-256 list. Help > Version History goes back to an older version if a new one gives you trouble. Preferences > Updates turns the check off or limits it to stable releases.
- **With or without Blender's libraries:** an install holds two programs, Blendity (its own code only, always starts) and Blendity-full (with Blender's libraries: faster ray tracing, denoising, OpenColorIO's views, Jolt physics, video export, Booleans, Decimate, EXR). Blendity starts first and hands over to the full one when its libraries load and you want them; offline, or with a library missing, it runs on its own code and says so. Preferences > Libraries is your choice; `--no-libs` starts without them once; when they are missing and you are online, Blendity offers to download them for this build.
- **On a small laptop** (the HP Envy 13-inch 2-in-1 class: 4 cores, 8 GB, Iris Xe) Blendity starts on the Low quality tier and says so, with High one click away (chapter 12).

1.4 Starting from the command line

```
Blendity --layout Learning            # start with the Learn tab beside the Scene view
Blendity --scene Assets/Scenes/SampleScene.scene
Blendity --scale 1.5                  # force the UI scale (the default follows the OS
     DPI)
Blendity --headless-screenshot out.png --cmd "select Cube" --cmd "shading rendered" --
    frames 30
```

The last form runs with no window: it executes console commands and writes a picture, which is how the tests, the figures in this guide and the comparison harnesses drive the editor.

1.5 Privacy

Blendity sends nothing about you or your work anywhere. The one network use it makes on its own is the optional daily update check. There is no telemetry and no cloud; the only learned model in the program is the optional, local AI Inbetweening download, which never changes a scene on its own (`docs/AI_DISCLOSURE.md`).

Learn

Welcome to Blendity: how this editor and the Learn tab work, installing and sharing, updates and older versions, with or without Blender's libraries.

Chapter 2

Getting around

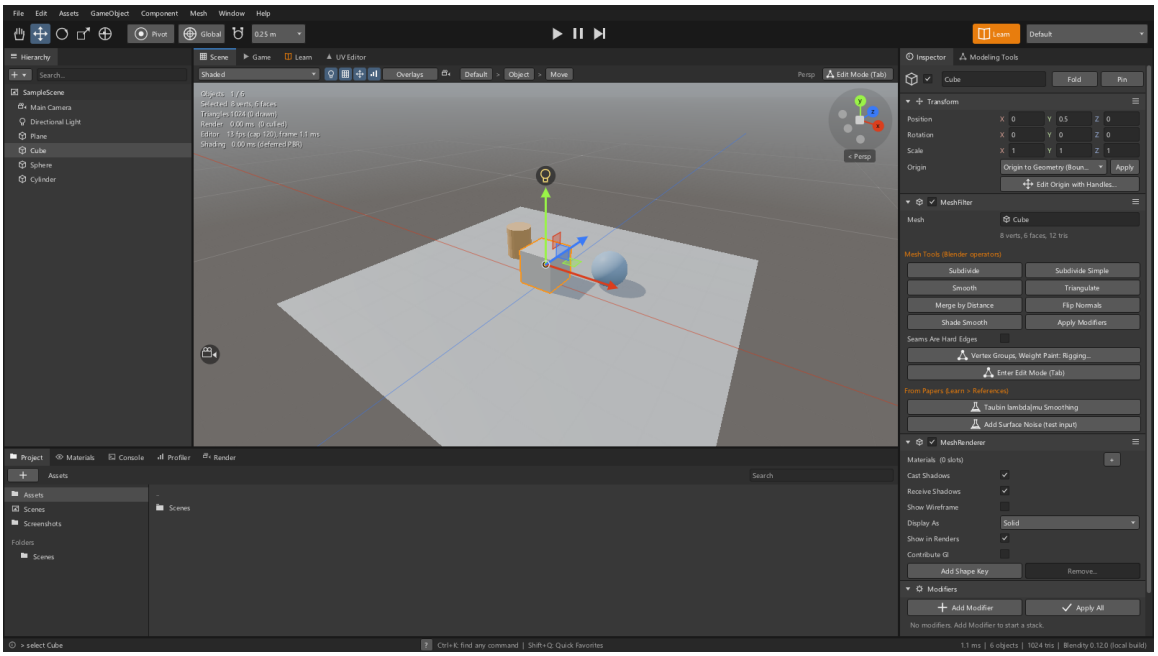


Figure 2.1: The Default layout: Hierarchy, Scene view, Inspector and Modeling Tools, Project and Console; the toolbar's tools, pivot, space and snap increment; Play, Pause and Step; the Learn button and the Layout dropdown at the top right.

2.1 The windows

Scene is the 3D view you edit in; **Game** shows what the active camera sees; **Hierarchy** is the objects as a tree; **Inspector** the selected object's components; **Project** the files in `Assets`; **Console** the messages and a command line. Under Window are the UV Editor, Render, Materials, Textures, Modeling Tools, Lighting, Rigging, Animation, Sequencer, Live Link, Editing, History, Profiler, Draw, Layers, Board, Avatar, My Tools and Learn windows. Drag a tab onto another tab bar to group it, onto a panel's edge to split it, out past the editor's edge for a window of its own on any display; middle-click a tab to close it; right-click it for more.

Coming from Unity

These are Unity's windows under Unity's names, docked the same way. **Modeling Tools** is the window Unity does not have: every modeling, sculpting, painting and drawing tool lives there (and in the Mesh menu), grouped by stage.

Coming from Blender

Scene view = 3D Viewport; Hierarchy = Outliner; Inspector = Properties editor (modifiers and materials are there too); Project = File / Asset Browser; Console = Info editor; the Layout dropdown = Workspaces. Areas split by dragging an edge as in Blender; tabs dock as in Unity.

2.2 Layouts for each stage

The Layout dropdown (top right) arranges the windows for a stage of the work and shows that stage's tools and tabs first, so the editor never feels like every tool at once. Nothing is ever gone: hidden tools still answer their keys and the palette, Window > Layouts > Show All Tools shows everything, and Save Layout… keeps your own.

Layout	Windows	Tools shown
Default	everything	everything
Reference & Planning	Scene, Game, Project, Board	create, reference images, cameras
Blocking Out	Scene, Modeling Tools	primitives, drawing, snapping
Modeling & Topology	Scene, Modeling Tools	Edit Mode, checks, curves and lattices
UV Unwrapping	Scene beside the UV Editor	Edit Mode, UVs
Texturing & Materials	Scene, Game, Materials, Textures	materials, textures, baking
Texturing, Stylized	Scene, Textures / Modeling Tools	surfaces, paint; strokes, scatter, sky
Lighting & Rendering	Scene, Game, Lighting, Render	lights, cameras, bakes
Editing	Editing, Render	developing a render
Rigging; Skinning & Weight Painting	Scene, Game, Rigging	bones, skins, IK, deformers; weights
Animation	Scene, Game, Animation along the bottom	animation, rigging, puppet posing
Drawing; Drawing with References	Draw, Layers, (Board)	the pen's pad, its layers, the board
Learning	Learn beside the Scene view	everything

In one area, Materials, Textures and Lighting fold into one Shading tab with a row of the three under it (Rigging & Animation and Tools fold the same way; Fold Groups in the tab menu turns it off). A tab strip whose labels don't fit shows the inactive tabs as icons, then lists the rest under ≫; the stage's hidden tabs sit under $+N$. The Scene view's bar reads *Layout* > *Mode* > *Tool*, each part a dropdown.

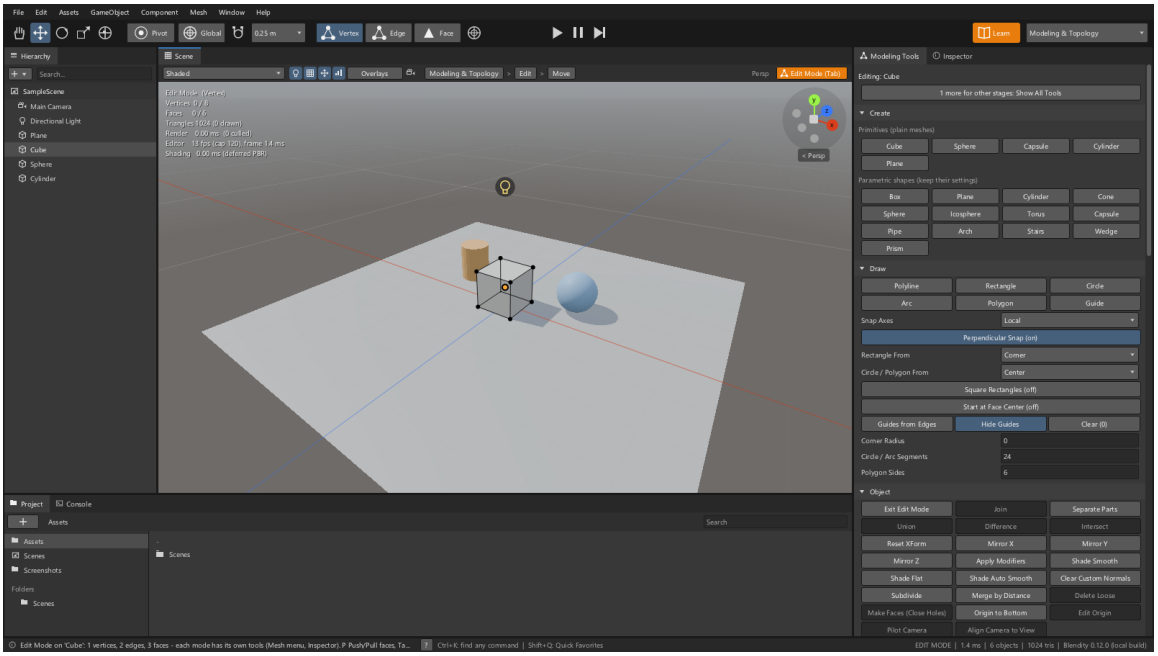


Figure 2.2: The Modeling & Topology layout in Edit Mode: the Modeling Tools window shows the Edit Mode sections (Select, Create & Extrude, Cut & Divide, Merge & Clean Up), the Draw tools and the checks.

2.3 Navigating

With Unity's mouse (the default feel): Alt+left drag orbits, middle drag pans, the wheel zooms, hold the right button and use W A S D (Q / E down and up, Shift faster) to fly, F frames the selection. Click the cones of the gizmo in the corner for front, top and side views, its centre for perspective or orthographic. Edit > Editor Feel switches the mouse navigation and the keys together to Unreal's, Blender's or Maya's (chapter 3).

2.4 Selecting and moving

Click to select, Ctrl+click to add, drag a box. Q W E R Y pick the View, Move, Rotate, Scale and Transform tools; Blender's G / R / S also work, with X / Y / Z to lock an axis and a typed number (the Blender keys for R and S are a preference, since R is Unity's Scale tool). Hold Ctrl to snap (the increment is the dropdown beside the magnet); Ctrl+Shift while moving drops the object onto the surface under the mouse; V snaps a vertex onto another. While dragging a gizmo arrow, type a number to move exactly that far.

2.5 Objects, components and the Inspector

As in Unity, everything in a scene is a GameObject with a Transform; what it does comes from its components (MeshFilter, MeshRenderer, Light, Camera, Rigidbody, Animation, modifiers, Camera Filters, ...). Create objects from the Hierarchy's right-click menu or the GameObject menu; drag a Hierarchy row onto another to parent it (in thirds: above, into, below). The Inspector shows each component's first seven

fields and keeps the rest under Show more; All Fields shows everything. Number fields take expressions: `2*pi`, `+=1`, `*=2` (relative to each object's own value), `L(0,10)` (spread evenly across the selection), `R(0,1)` (random per object); with several objects selected a change applies to all of them. A field that names another object takes a typed name or a row dragged from the Hierarchy; an asset field takes a file dragged from the Project window. Right-click a field's name for Reset to Default or Insert Keyframe.

2.6 Origins, Play, undo

GameObject > Set Origin moves the pivot to the bounds centre, the median, the centre of mass, the bottom centre, a point, the edit selection or the view pivot without moving anything on screen; Edit Origin (Handles) moves only the origin with the gizmo and snaps it to vertices, edges and faces. Ctrl+P plays the scene; changes made while playing are thrown away on Stop, as in Unity. Ctrl+Z / Ctrl+Y undo and redo the scene, materials, textures and clips as one step per operation; Window > History lists every step and a click goes back to it.

2.7 Finding anything: the palette, favorites, the Wheel

Ctrl+K (or F3, Blender's key) opens the **command palette**: type a few words and every action, window, layout, creatable object and console command that matches is listed with its shortcut; Enter runs it, even a tool the current layout hides. Right-click a row (or any menu row) for its small menu: **Quick Favorites** (Shift+Q, one list per layout), **the Wheel** (Space: one radial menu for where the mouse is, with a set per mode you fill from any menu), **My Tools** (a window of your own panels, each a grid of commands), **Assign Shortcut...** (the next key you press binds it). Preferences > UI file saves the whole arrangement (layout, windows, feel, keymap and your shortcuts, panels, the Wheel's sets, favorites, theme, scale) as one `.blendity-ui` file.

2.8 The status bar, toasts and the job tray

The status bar says what the mouse and keys do where the mouse is and offers Undo for the last action; the ? beside the hint opens the Learn lesson for what you are doing. A warning is a toast over the Scene view for three seconds, with a Fix button when a command puts it right (stop Play, cancel the job, unwrap first); a button that cannot work now is dim and its tooltip says why. A long job (a bake, a remesh, a render, an export) shows in the status bar's right slot with its progress, the time run, an estimate left and Cancel; a toast says when it ends. The results of checks (UV Report, Scene Statistics, Motion Check) open in a Report dialog.

2.9 Windows on other screens and Tablet Mode

Drag any tab out past the editor's edge for a window of its own where you let go; right-click a tab for Move to New Window, Move to Display N or Move to Main Window. A Render or Game tab in its own window can be Render Only and Full Screen: a second monitor showing the render large. The Scene view, the Game view and the Inspector can be duplicated into a new window; a second Inspector has a Pin that keeps one

object while the selection moves on. **Tablet Mode** (the tab menu, or Preferences > Pen and touch: Auto turns it on when a 2-in-1's keyboard folds away) makes the targets half as big again, and then fingers navigate (one orbits, two pinch, pan and twist) while the pen selects, moves, draws and paints (chapter 11).

2.10 Themes

Preferences > Theme: Dark, Light, Emiliano Light, Emiliano Dark (the palette of the author's site: paper, charcoal and an amber accent) and High Contrast (white on near-black with every text at WCAG's 4.5:1 or more [46] and a yellow focus ring; the tests measure every theme's ratios).

Learn

The Editor Layout, *Navigating the Scene View*, *GameObjects, Components and Objects*, *Hierarchy, Parenting and the Scene Graph*, *Move, Rotate, Scale*, *Customizing the Editor*, *Undo, Selection and Memory*.

Chapter 3

Coming from another program

Blendity has two kinds of users by design: Unity users who expect Unity's navigation, its W / E / R tools and an Inspector, and Blender users who expect to model with the keyboard. The rule that resolves the clashes (`docs/USABILITY.md`) is: **move the view and pick tools as Unity does; change geometry as Blender does (a verb key, the mouse or a typed number, click to confirm, F9 to adjust); settings in the Inspector**. Nothing a Unity user already knows changes by default, and everything a Blender user reaches for while modeling works. The first run asks where you come from and sets the keys and the feel; Edit > Preferences > Keymap starts from any of seven presets and rebinds anything; `keymap table` in the console prints every action's chords under every preset.

3.1 The editor feel

Edit > Editor Feel switches the keys *and* the Scene view's mouse navigation to another program's:

Feel	Orbit	Pan	Zoom / dolly	Look + fly	Its own keys
Unity	Alt+LMB	MMB, Alt+Ctrl+LMB	Wheel, Alt+RMB	hold RMB + WASD	the Unity table below
Unreal	Alt+LMB	MMB, LMB+RMB	Wheel, Alt+RMB	hold RMB + WASD	Alt+P play, End drop to floor, Alt+G/H/J/K views
Blender	MMB	Shift+MMB	Wheel, Ctrl+MMB	–	G/R/S, A, X, Shift+D
Maya	Alt+LMB	Alt+MMB	Wheel, Alt+RMB	–	Q/W/E/R, F8, F9–F11

Window names, layouts and the Y-up axis stay Unity's whatever the feel.

3.2 From Unity

What carries over

Everything on the surface: the windows, Alt-orbit, Q W E R Y, F to frame, Ctrl+D duplicate, Ctrl+P play, Ctrl+Shift+N empty, Ctrl+1…9 windows, the Inspector with multi-object editing, drag-and-drop of assets, the Project window, Play mode that reverts, the Animation window (dopesheet, Curves, Record, Add Property, K keys every track, comma and full stop step keys), Humanoid retargeting as Mecanim reads a T and an A pose, `.anim` clips, `.unitypackage`, prefabs, scenes and `.mat` files read and written, LOD Groups, colliders, Light modes (Realtime / Mixed / Baked),

Contribute GI, lightmaps and probe volumes, post-processing as Camera Filters.

What is new is **Edit Mode** (Tab) with Blender's operators (chapter 5), the **Modeling Tools** window, **F9** to adjust the last operation, materials evaluating Blender's Principled BSDF with a layer stack of textures instead of a shader graph, two render engines in F12, and the **Learn** tab. What is not there: C# scripts (Play has a few built-in behaviours and the Sequencer's triggers) and the Animator state machine (the Sequencer's tracks and triggers do the job). A Unity scene opened here looks as its Game view does, and Live Link sends your work back into a Unity project as you edit (chapter 9).

3.3 From Blender

What carries over

The data and the tools: the mesh with n-gons, per-corner UVs and seams; Tab, 1 / 2 / 3, G and the axis keys with typed values, E extrude, I inset, Ctrl+B bevel, Ctrl+R loop cut, K knife, Alt+click loops and Ctrl+Alt+click rings, F9, O proportional editing, Shift+N normals, the modifier stack with the same header toggles, the Principled BSDF, LSCM and Smart UV, the Hosek–Wilkie sky, Filmic and AgX (with OpenColorIO), Cycles-like path tracing, the Graph Editor's tangent modes, bones and armatures, automatic weights, the NLA's idea as sequences, every export format with Blender's options.

Pick the **Blender** preset and feel: middle-mouse orbit, R rotates and S scales, A selects all, X deletes, Shift+D duplicates, F fills, M merges, J connects, L selects linked, . frames, Shift+I keys. Under the Unity preset R is the Scale tool (Edit > Blender Transform Keys moves Scale to T), E is the Rotate tool and Ctrl+E extrudes, Alt+F fills, Alt+M merges. What is different in kind: windows dock as Unity's and layouts replace workspaces; the Inspector replaces the N panel and the redo panel (tool numbers live in F9 or the drag helper, not the Inspector); the axes are Unity's (Y up, left-handed: OBJ and FBX mirror X on the way out as Unity's importer does); there is no node editor (chapter 6 maps Blender's texture and converter nodes to layers), no Python (the console), no `.blend` except through your installed Blender (`import file.blend`), and Live Link sends the scene into Blender live.

3.4 From Maya

Pick the **Maya** preset and feel: Alt+LMB orbit, Alt+MMB pan, Q W E R tools, F8 toggles components (Edit Mode), F9 / F10 / F11 vertex / edge / face (so Adjust Last moves to Ctrl+F9 and the Render window leaves F11), S sets a key, B soft select (proportional editing), Ctrl+Shift+X Multi-Cut (the knife), Ctrl+M collapse, Z / Shift+Z undo and redo. Maya's Outliner is the Hierarchy, the Attribute Editor the Inspector, the Hypershade the Materials and Textures windows, the Graph Editor the Animation window's Curves view.

3.5 From 3ds Max

Pick the **3ds Max** preset: Q W E R, Z frames, Tab enters the editable poly's sub-objects (Edit Mode), 1 / 2 / 4 vertex / edge / polygon, Ctrl+V clones, Shift+E extrude, Shift+I inset, Ctrl+Shift+C chamfer (bevel), Alt+C cut (knife), Ctrl+Shift+E connect, Ctrl+Alt+C collapse, Ctrl+D deselects. The modifier stack is the Inspector's modifier list, bottom to top as Blender's.

3.6 From SketchUp

Pick the **SketchUp** preset: Space selects, M moves, Q rotates, S scales, L line, R rectangle, C circle, A arc, Shift+P polygon, P Push/Pull, F offset (inset), E erases, Shift+Z zoom extents, Shift+Space the Wheel. The **Draw** tools draw onto faces or the ground with SketchUp's inference (the axes, parallel and perpendicular edges, midpoints, face centres); a closed shape on a face is cut into it, ready to **Push/Pull** with SketchUp's semantics (ADR 0004, [110]): pulled, it extrudes; pushed through, it makes a hole; pushed flush with another face, it merges; Ctrl keeps the original face. Follow Me is Follow, the Tape Measure's guides are Guide and the X / Y / Z construction lines; groups and components are objects and child objects; **N-gon Mode** treats each flat region as one face as SketchUp does.

3.7 From Unreal

Pick the **Unreal** preset and feel: Unity's keys with Unreal's on top (Alt+P plays, Ctrl+W duplicates too, End drops the selection to the floor, Alt+G/H/J/K the perspective, front, top and side views, LMB+RMB pans). The Outliner is the Hierarchy, the Details panel the Inspector, the Content Browser the Project window, the Sequencer is here under the same name (clips, cards, camera cuts, sound), Post Process Volumes are Camera Filters. Live Link sends the scene into a level through the Unreal bridge (lights, cameras, LODs and UCX collision as Unreal names them), and Send to Blendity brings a level back.

3.8 From Photoshop, Substance Painter and SketchBook

Pick the **Adobe** preset: Unity's keys with Photoshop's brush keys on top [86]: B enters Paint Mode (and leaves Sculpt), E toggles the eraser (or Carve in Sculpt), V is the Move tool, H the Hand tool, [/] the brush size (in every preset), Shift+[/] its opacity, Ctrl+T Free Transform, Space held pans, Ctrl+0 fits. The **Draw tab** works like SketchBook Pro's canvas [109] with Photoshop's 27 blend modes, clipping and layer masks, rectangle and lasso selections and Illustrator's shapes and Pen (chapter 11). The **Textures** window's surfaces are Substance Painter's pipeline [87]: bake mesh maps, fill layers with mask stacks reading them, smart masks and smart materials, a material brush that paints colour, roughness, metallic, height and emission at once, and a texture set exported in the glTF, Unity, URP, HDRP, Unreal, Poiyomi or lilToon packing (chapter 6). The **Editing** tab develops a render as Lightroom does. The **Board** is a PureRef-style reference board [74].

3.9 The shortcut table

The table below is the Unity preset beside Blender's own keys; `keymap table` prints every preset.

	Blendity (Unity preset)	Blender
Orbit / Pan / Zoom	Alt+LMB / MMB / Wheel or Alt+RMB	MMB / Shift+MMB / Wheel
Fly	hold RMB + WASD, Q/E, Shift	Shift+‘
Frame selection	F	Numpad .
Views, Persp / Iso	the scene gizmo's axes / centre	Numpad 1 / 3 / 7, Numpad 5
Create objects	right-click in the Hierarchy	Shift+A
View / Move / Rotate / Scale / Transform	Q / W / E / R / Y	– / G / R / S / Transform tool
Duplicate / Delete / Rename	Ctrl+D / Del / F2	Shift+D / X / F2
Play / Pause / Step	Ctrl+P / Ctrl+Shift+P / Ctrl+Alt+P	–
Edit Mode / vertex / edge / face	Tab / 1 / 2 / 3	Tab / 1 / 2 / 3
Grab / rotate / scale (modal)	G, X / Y / Z, a typed value	G / R / S
Extrude / Inset / Bevel	Ctrl+E / I or Ctrl+I / Ctrl+B	E / I / Ctrl+B
Select edge loop / ring	Alt+click / Ctrl+Alt+click	Alt+click / Ctrl+Alt+click
Loop cut / Knife / Connect	Ctrl+R / K / J	Ctrl+R / K / J
Push/Pull (face mode) / Push Through	P / Alt+P	–
Bridge	Ctrl+Shift+B	Bridge Edge Loops
Fill / Merge at center / Recalculate normals	Alt+F / Alt+M / Shift+N	F / M / Shift+N
Dissolve / Triangulate / Tris to quads	Ctrl+X / Ctrl+T / Alt+J	Ctrl+X / Ctrl+T / Alt+J
Proportional editing	O	O
Adjust Last Operation	F9	F9
Set a key / Record / prev, next key	Shift+K / Ctrl+Shift+K / , .	Shift+I / – / Down, Up
UV Editor / Unwrap	Ctrl+9 / U	UV Editing workspace / U
Render / Render window / Render Animation	F12 / F11 / Ctrl+F12	F12 / F11 / Ctrl+F12
Palette / Favorites / the Wheel	Ctrl+K or F3 / Shift+Q / Space	F3 / Q / –
Learn	F1	–

Learn

Navigating the Scene View (the shortcut sheet), *Coordinate Systems and Handedness*, *Edit Mode: Modeling Like Blender*, *Customizing the Editor*.

Chapter 4

Reference, planning and blocking out

4.1 Reference images and the board

GameObject > Reference Image… puts a picture into the scene as an unlit plane facing the front, sized to the picture, kept out of renders and shadows (`refimage <path> front|side|top` picks the view). A picture from outside the project is copied into `Assets/Textures/References`, so the scene keeps it on another machine. The **Board** (Window > Board, or the Drawing with References layout) is a reference board in PureRef's way [74]: drop images from the system, paste a screenshot with Ctrl+V or drag them from the Project window onto an endless canvas; move, scale and turn them by their handles; it saves itself to `Assets/Boards` and opens again next time. Boards > Export… writes it as PNG, JPEG or PDF.

Example: model from a front and a side drawing

1. Layout > Reference & Planning.
2. GameObject > Reference Image…, pick the front drawing; for the side drawing type `refimage <path> side` in the Console.
3. Scale and move the planes so the drawings line up (the Inspector's Scale field takes `*=2`).
4. Switch to Blocking Out and start with primitives over them.

4.2 Cameras and shots

Add a Camera (GameObject > Camera: it is placed at the view, looking its way) and pick it in the Game view's camera menu; selecting a camera shows a **Camera Preview** in the corner of the Scene view. **Pilot Camera** flies the Scene view as that camera; composition guides (thirds, golden ratio, safe areas) draw over its frame. Physical camera settings (focal length, sensor presets, f-stop, focus distance, blades, shift) are in the Camera's Inspector; the focus eyedropper focuses on the surface you click, and Pick Focus Point keeps a world point in focus. The scene's **Active Camera** is what F12 and the Game view use.

4.3 Blocking out

GameObject > 3D Object is in every layout: cube, sphere, icosphere, cylinder, cone, torus, plane, quad, capsule and the Utah teapot [31]. GameObject > Shapes (Parametric): box, plane, cylinder, cone, sphere, icosphere, torus, capsule, pipe, arch, stairs, wedge and prism keep their settings in the Inspector and rebuild as you change them, like ProBuilder's shapes [121]. Right after adding a primitive, the **Adjust Last** panel

at the bottom left of the Scene view (F9) changes its size, segments or rings as one undo step.

Drawing, SketchUp-style. The Draw tools (Polyline, Rectangle, Circle, Arc, Polygon) draw onto faces or the ground with snapping to corners, midpoints, edges, face centres and the drawing plane's axes; X / Y / Z lay construction lines; Ctrl+click lays a guide along an edge; rectangles come from a corner, the centre or three points; circles from the centre, a diameter or three points. A closed shape on a face is cut into it, ready to **Push/Pull** (P): move the mouse or type a distance, click to confirm. Pushed through, it makes a hole, imprinted on whatever face it exits through; pulled onto another face, it joins it; Ctrl keeps the original face. [110, 111, 108]

Booleans (with Blender's libraries: the Manifold solver). Select the cutters, then Ctrl+click the object to cut last; Mesh > Boolean (Union, Difference, Intersect) applies at once or adds a live modifier whose cutter stays editable as a wire outline. **Join** (Ctrl+J) combines meshes; **Separate** splits off the selection or each loose part. From UModeler and Plasticity: **Follow** (sweep a face along a path), **Spin / Lathe**, **Slice**, **Shell**, **Thicken**, **Draft**, a **Radial Array**; **Smart Fill** closes holes and wire loops. Hold Ctrl for grid snapping; Ctrl+Shift while moving drops an object onto the surface under the mouse.

Example: a doorway in a wall

1. GameObject > Shapes (Parametric) > Box; in the Inspector make it 4 × 3 × 0.2.
2. Select the box, Rectangle (Modeling Tools > Draw), and draw a rectangle about 1 × 2.1 on its front face from the bottom edge (hold Ctrl to snap; the length shows as you draw).
3. Press P over the rectangle and push it through the wall: the doorway is cut.

4.4 Drawing in perspective, building from the construction

For a drawing pad or a pen: Draw > Sketch (pen) draws over the Scene view with SketchBook's aids under Draw > Guides [109]: a ruler, an ellipse, French curves, symmetry, and 1-, 2- and 3-point perspective with draggable vanishing points. When the pen lifts the stroke is corrected to what it meant [19]: split at its corners, each piece a line, an arc, an ellipse or a curve, pulled vertical, horizontal or onto a vanishing line, its ends joined to other strokes'. The 3-point guide *is* the camera: orbit and its points move; drag a point and the camera turns to put it there. A **canvas** can be a plane in the scene (the ground, a face, a wall through a point) and a stroke on it sits in the scene from every view; a line along the plane's axis becomes a construction line. Draw > Build from Sketch makes solids from the strokes that fix them, 3-Sweep's way [6]: a box from three lines at a corner, a cylinder or cone from two circles, a sweep from two circles and a path, a face from a closed outline and its Push/Pull from a line along its normal, with their numbers in F9. Draw > Sketch over the Model keeps a drawing true from its view and traces a stroke into a cut or a curve; `sketchlift` lifts a whole scaffold drawn under the 3-point guide into a wire object [13]. Nothing is guessed: the model is built by the artist's own steps.

Example: a box from a drawing

1. Draw > Sketch (pen); `sketchcanvas ground`. Draw two lines from one point along the red and the blue axis (they snap along the axes and become construction lines).
2. `sketchcanvas plane z 0 0 0` (a wall through the origin) and draw the upright from the same corner.
3. Draw > Build from Sketch > Box: the box stands on the three lines; F9 corrects its lengths.

Learn

Cameras and Projection, *Points, Lines, N-gons and Booleans*, *Move, Rotate, Scale*, *Drawing in Perspective*, *Papers*.

Chapter 5

Modeling, sculpting and UVs

5.1 Edit Mode

Tab enters Edit Mode; 1 / 2 / 3 pick vertex, edge and face selection. The tools follow the selection mode, as in ProBuilder and Blender:

Do	Unity keymap	What it does
Extrude	Ctrl+E	pull new geometry out of the selection; a lone vertex grows an edge, an edge a face
Inset	I / Ctrl+I	a smaller face inside each face (or the selection as one region)
Bevel	Ctrl+B (wheel: segments)	round or chamfer edges, with a profile from concave to convex
Loop cut	Ctrl+R over an edge	cut a ring of new edges round the mesh
Knife / Line	K	split a face between two points with endpoint, mid-point and on-edge snapping
Bridge	Ctrl+Shift+B	join two faces or two holes with a tube
Merge at center / Fill	Alt+M / Alt+F	collapse vertices, close holes (Smart Fill on open loops)
Dissolve / Connect	Ctrl+X / J	remove edges keeping the surface; cut between two vertices
Select loop / ring	Alt+click / Ctrl+Alt+click	edge loops and rings; face loops in face mode
Push/Pull / Push Through	P / Alt+P	SketchUp's face operator; a hole along the normal

Every modal operator behaves the same way: start with the key; move the mouse or type a number (X / Y / Z lock an axis, twice for the local axis; Shift+axis locks a plane; Ctrl snaps; Shift is fine control; during a bevel the wheel sets the segments, or] / [without one); a click or Enter confirms; Esc or a right-click cancels leaving no undo step; then **Adjust Last Operation** (F9) changes the values afterwards as the same step and remembers them as the tool's defaults. A hint bar at the bottom of the Scene view lists the keys while any of them runs. **Proportional editing** (O)

moves nearby vertices with a falloff. **N-gon Mode** treats each flat region as one face, as SketchUp does. Shift+drag a gizmo handle extrudes first.

Modifiers stack non-destructively, top to bottom, as in Blender: Array (with Fit Length / Fit Curve), Bevel, Boolean, Decimate, Mirror, Screw, Solidify, Subdivision Surface [5], Triangulate, Wireframe, Weld; and the deformers Cast, Curve, Displace, Lattice, Mesh Deform, Simple Deform and Smooth. Each has Edit Mode / Viewport / Render toggles and a header as Blender's; drag a modifier's name to move it; Apply works on one modifier ignoring those above it. **Curves and lattices**: GameObject > Curve makes Bezier, circle and path curves; a Lattice is a box of points that squashes the mesh inside it; their points take the same keys as vertices.

Checks. Select Overlapping and the Z-fighting check find faces on top of each other; Delete Loose removes stray vertices; Recalculate Normals (Shift+N) faces everything outwards; Auto Smooth shades by angle; Select Similar and Select by Material; Mesh > From Papers has the tools built from research papers (Taubin smoothing [43] and others).

Example: a rounded table top

1. Create a Cube, scale it to $1.6 \times 0.05 \times 0.9$.
2. Tab into Edit Mode, 2 for edges, Ctrl+Alt+click one of the four vertical edges (an edge ring).
3. Ctrl+B and drag; scroll for segments. F9 to set the width exactly.

5.2 Sculpting

Modeling Tools > Sculpt (`sculpt on`) is ZBrush's workflow on the mesh under the pen [106]: brushes (Clay Buildup, Draw, Inflate, Smooth, Grab, Snake Hook, Crease, Flatten, Scrape, Pinch) with Mirror symmetry and Radial, a lazy mouse, image alphas and stroke types (Dots, Drag Dot, Drag Rect, Freehand, Spray); masks (Ctrl paints, Ctrl+I inverts) that keep a brush off a region; polygroups in the eight colour-universal colours [33] and hiding (H / Alt+H) to work on a part alone; a stroke one light undo step; the pen's pressure, tilt and eraser end (the [/] keys step the size). Four ways to add resolution: **Dynamic Subdivision** (a Catmull–Clark preview the brush edits through; the base mesh stays light), **DynaMesh** (the shell retopologized evenly from its winding number [20, 12]; two shapes merged into one skin), **Quad Remesh** (QuadriFlow [18] on a worker: a clean quad mesh at a target count, with Blender's Eigen) and **Dyntopo** (Sculptris Pro [107]: the mesh tessellated under the brush as it moves). The pipeline order that keeps a model clean: *sculpt* > *remesh* > *unwrap* > *bake mesh maps* > *paint*; a remesh after painting would throw the paint's UVs away.

5.3 UV unwrapping

Layout > UV Unwrapping puts the **UV Editor** (Ctrl+9) beside the Scene view.

1. In Edit Mode select the edges where the mesh should be cut open and Mark Seam (or UV > Seams from Sharp Edges).

2. Select everything and press U to Unwrap (LSCM, as Blender's [25]). Smart UV projects by angle; Part Unwrap finds charts by parts with ABF++ [41, 49]; cube, cylinder, sphere and view projections are there too.

3. In the UV Editor move, rotate and scale islands with the gizmo (W / E / R); Pack fits them in the square; Avg Scale evens them; the Stretch overlay shows where a texture would distort; Sync sets Edit Mode's faces from the UV selection. After Unwrap, Smart UV or Pack, F9 changes the margin.

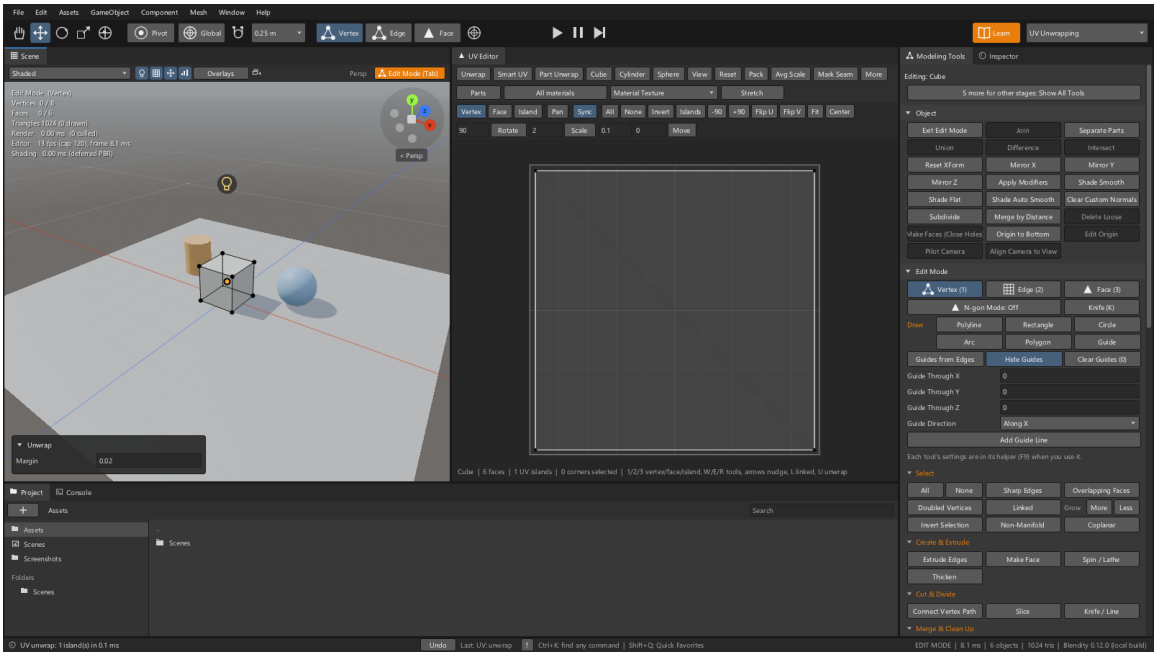


Figure 5.1: The UV Unwrapping layout: Edit Mode on the cube, the UV Editor's toolbar (Unwrap, Smart UV, Part Unwrap, the projections, Pack, Mark Seam; vertex / face / island selection; the transforms) and the Modeling Tools beside it.

Example: a crate with a clean texture layout

1. Create a Cube; Edit Mode, select all, UV > Cube Projection.

2. Set the UV Editor's background to UV Grid (Blender's test image) to check that the squares stay square.

Where it comes from

Edit Mode's operators follow Blender's BMesh operators and editors [71, 90]; Push/Pull, inference and Follow are SketchUp's [110]; Follow, Lathe and Slice are UModeler's [111]; Shell, Draft, Radial Array and the construction planes are Plasticity's [108]; the parametric shapes are ProBuilder's [121]; the sculpt brushes, masks and remeshers follow ZBrush and Sculptris [106, 107]; the theory is in [54] (meshes, texture coordinates). Learn: *Meshes: Vertices, Edges, Faces*, *Edit Mode: Modeling Like Blender*, *Modifiers vs Components*, *Advanced Modeling Tools*, *Bevel, Bridge and Push Through*, *Sculpting*, *UV Mapping*.

Chapter 6

Materials, textures and painting

6.1 Materials

Materials are Unity-style, evaluating Blender's **Principled BSDF**: base colour, metallic, roughness, specular, normal and emission, with tiling and offset, surface types Opaque, Cutout, Transparent, Glass and Volume (Fog), the folded Principled: Subsurface, Sheen, Coat group (skin, cloth, lacquer; Anisotropic for brushed metal and hair), and shading models PBR, Toon (five styles with painted shadow, highlight and rim control, shade maps, ramps, a face shadow map, inner lines and hatching, an Outline component; the lilToon group for VRChat avatars [77]) and Hand-painted. The **Materials** window shows every material as a sphere (a software-shaded preview, Blinn–Phong [36]): drag one onto an object (the face under the mouse picks the slot) or onto a face in Edit Mode; Assign to Selection does it for the selection. Material assets (`Assets/Materials/<name>.mat`) are shared by every object that uses them, as in Unity; a Unity project's `.mat` loads by its shader and is never written over. **Axis materials** pick top, side and bottom materials by slope without UVs. **Fog**: Surface Type Volume (Fog) makes a closed mesh's inside fog with Blender's Principled Volume fields; F12 with path tracing draws it truly.

6.2 Procedural textures: Blender's nodes as layers

Procedural textures (`Assets/Textures/<name>.ptex`) are a stack of layers, like Blender's modifier stack: generators (Brick, Wood Floor, Tiles, Voronoi, Noise with Musgrave's types, Perlin Noise [35], Wave, Magic, White Noise, Gabor, Checker, Gradient, the anime background generators) and filters (Blur, Levels, Warp, Edge Wear, Color Ramp, Map Range, Clamp, Math, RGB Curves, Separate Channel, Posterize, Cavity), each with a blend mode (Blender's Mix modes, Photoshop's too). One texture gives albedo, normal, roughness and height maps and stays sharp at any distance with Live Shading; its Physical Size is the metres one tile covers. The **Textures** window makes and edits them: New lists presets by theme; every pattern has a folded Mapping group (offset, rotation, scale, Seamless); Add Layer has a search; Alt+click a layer's name to see the stack up to it; right-click any field's name for Reset to Default. Blendity has no node canvas: each Blender node is a layer, a layer's option or a material field.

Blender node	In Blendity
Texture Coordinate, Mapping	the material's Mapping (UV, Box / triplanar, Generated); a layer's Mapping group
Noise, Voronoi, Wave, Magic, White Noise, Gabor, Checker, Brick, Gradient	the layers of those names
Image Texture	the Image layer, or a material's Base Map
Sky Texture, Environment Texture	the Lighting window's Environment; the Sky Gradient layer
Mix Color	a layer's Blend
RGB Curves, Invert, Hue/Saturation/Value, Bright/Contrast	the RGB Curves, Invert, Hue / Saturation and Levels layers
Color Ramp, Map Range, Clamp, Math, Separate RGB / HSV	the layers of those names
Bump, Normal Map, Displacement	a texture's Height as its normal; the material's Normal Map; the Displace modifier
Principled BSDF, Glass, Transparent, Principled Volume	the material; Surface Type Glass, Transparent, Volume (Fog)
Toon BSDF, Shader to RGB	Shading Model Toon
Mix Shader, Light Path, Attribute, Geometry	not yet (an axis material mixes three materials by slope)

Example: a brick wall material

1. Window > Textures > New > Bricks. Adjust the brick layer's fields; it already has an Edge Wear layer: raise it for worn corners.
2. Select the wall and press Assign to Selection: albedo, normal and roughness are set, with the tiling.

6.3 Painting the surface: the Substance Painter pipeline

In order [87]:

1. GameObject > Bake Mesh Maps… (or the Textures window's Bake All Maps): the mesh's own normal, position, curvature, ambient occlusion and thickness baked into its UV layout on a worker, under `Assets/Textures/MeshMaps`.
2. GameObject > New Surface: a texture in the object's own UV space whose layers write colour, height, roughness, metallic and emission; **Fill** layers with a **mask stack** (a mesh map through levels, Noise, Blur, Levels, an Anchor to another layer's mask), the **smart masks** (edge wear, dirt, dust, metal edge wear, position gradient, thickness, Ground Dirt, Cavity, a Material ID Mask), **Group** layers. Editing a layer redoes only the layers above it inside the edited rectangle.
3. **Paint Mode** (Modeling Tools > Paint; B under the Adobe preset): a material brush paints colour, roughness, metallic, height and emission at once by projection; the far side and anything behind are left alone; the pen's eraser end erases; Mirror X / Y / Z; Bucket Fill fills the group, island, material or object you click, or a gradient; Stencil holds an image over the view; a stroke is one light undo step. `paint 2d on` paints on the Textures window's canvas instead.
4. **Smart materials**: the Textures window's shelf applies a built-in (Worn Steel, Painted Metal, Rubber, Wood, Plastic, Concrete, Gold) on top of the surface, where its generators read *this* object's mesh maps; Save as Smart Material keeps a group of your own.
5. **Export Textures…** (the Textures toolbar): the texture set packed as glTF, Unity, URP, HDRP, Unreal, Poiyomi or lilToon reads it, or separate maps.

The hand-painted look. When the light should live in the texture: `paint bakelight` bakes the mesh maps' occlusion and a key light into a Multiply layer to paint over; the Paint section's Tip (Round, Flat turned to the drag, Bristle), Mode (Paint, Smudge, Blend), Wet (the brush carries the colour it went over [3]), Jitter and grain make the brush behave like paint; the palette row keeps swatches and shades; set the material's Shading Model to Hand-painted so every renderer shows the texture as painted.

6.4 Stylized environments and the anime look

Modeling Tools > Strokes paints geometry: a click on a surface adds a card, a drag with Ribbon a strip, into one Stroke object whose cards shade by their own facing, by the surface's normal, or away from a Canopy centre so a bush of flat cards reads as one round volume; `strokeobj grass` drops blades onto the ground. Modeling Tools

> Scatter places instances of a named object by density with a painted mask. The World's Sun Disc, Horizon Band and Fog finish the look in every renderer alike. For the 2D look: Camera Flatten squashes a character's depth toward the camera as Guilty Gear Xrd does [30]; View Angle Shapes brings shape keys in by the camera's angle; Anime Face Normals gives a head a proxy so toon shadows fall as drawn shapes; a Lattice can face the camera; the Anime and Manga camera filters add line art, halftone and paper.

6.5 Bake Textures

Bake Textures turns everything an object's materials do into plain images on new UVs, so other programs show what Blendity renders; on export, Bake Textures does it for the file.

Where it comes from

The Principled BSDF and the texture nodes are Blender's [71, 90]; the layer stack, mesh maps, smart masks and smart materials are Substance Painter's [87]; the painterly brush simplifies IMPaSTo [3]; the toon look follows Guilty Gear Xrd [30], MToon [84] and lilToon [77]; the theory is in [54] (texture mapping, procedural textures) and [52] (materials, textures and shaders). Learn: *Materials and Textures*, *Blender's Shader Nodes in the Textures Window*, *Fog with a Volume Material*, *Working Faster with Textures and Materials*, *Painting the Surface*, *The Hand-painted Look*, *Stylized Environments*, *Flatten to the Camera*, *Shapes by the Camera's Angle*.

Chapter 7

Lighting and rendering

7.1 Lights, the world and the views

Lights are Directional, Point, Spot and Area, with an optional colour temperature in kelvin [23]; Aim at Point turns a light to shine at the surface you click. The Scene and Game views rasterize (deferred PBR with sun shadows and image-based lighting), so editing stays fast. The world is a gradient, Blender's Hosek–Wilkie physical sky [17] driven by the first Directional Light, an HDRI or a colour, with a sun disc, a horizon band and distance fog; the view transforms are Standard, Filmic, ACES, Unity's ACES (the curve Unity's post-processing applies [16]) and, with OpenColorIO, AgX and Blender's others. The sky, fog and ambient light are edited in one place, the **Lighting** window.

7.2 F12 and the Render window

F12 renders the active camera with either engine: **rasterized** with supersampling, or **path traced** (Cycles-like: soft shadows, bounced light, glass, fog, depth of field, with Embree and OpenImageDenoise when Blender's libraries are present, else Blendity's own BVH and À-Trous denoiser). The **Render** window (F11) has resolution presets, the device (CPU, GPU Compute on any Vulkan GPU, OptiX on NVIDIA, or GPU + CPU together: path tracing picks the faster device by itself and never overruns a small GPU's memory), samples, path guiding, Film > Transparent, and saves PNG, JPEG, HDR or EXR. Render Camera Sequence renders every camera marked In Sequence; Render Animation (Ctrl+F12) renders every frame of the sequence or the selected clip as numbered PNGs and an H.264 MP4 beside them (ProRes and a VRChat-ready preset too; the dependency-free build offers Blender's libraries or an `ffmpeg` on the PATH); Live Preview re-renders a quick preview as you edit; the image pans with a left drag. MoonRay, DreamWorks' renderer [73], is an optional download run as a program of its own.

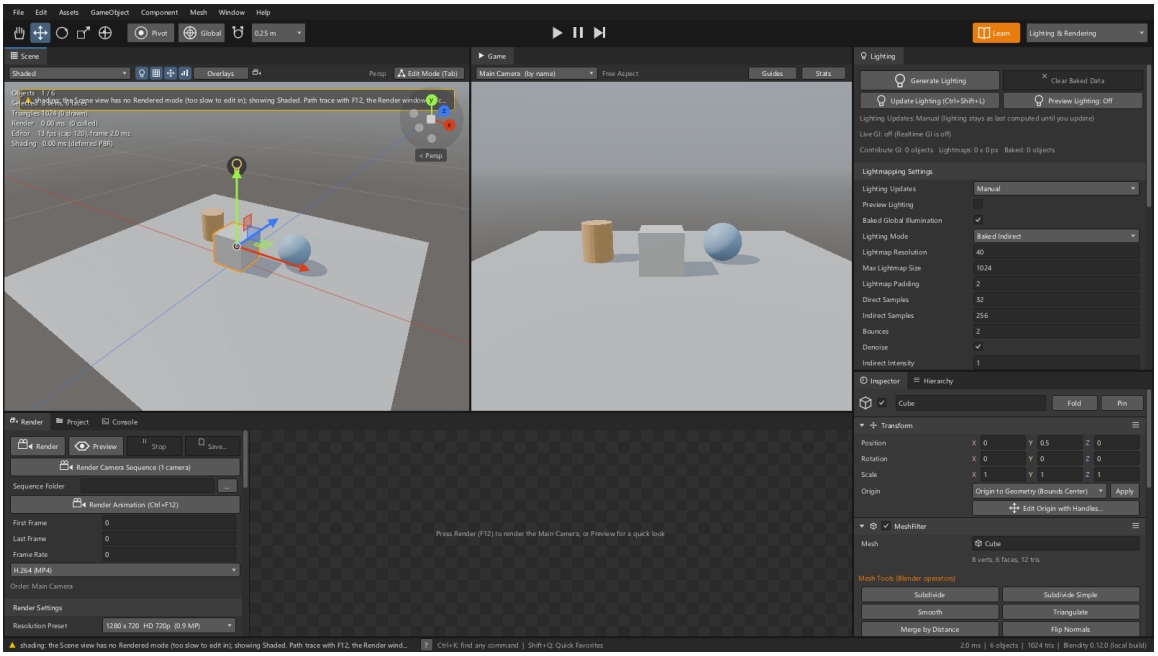


Figure 7.1: The Lighting & Rendering layout: the Scene view, the Game view through the Main Camera, the Lighting window (Generate Lighting, the lightmapping settings) and the Render window with its presets.

7.3 Baked, mixed and realtime light

The short version for a scene that doesn't lag, as Unity does it [122]:

1. **Static objects:** select everything that never moves and tick MeshRenderer > Contribute GI (Mark Static Meshes does it for a world). Lightmap UVs are made for you.

2. **Lights:** set each light's Mode: *Baked* (everything baked), *Mixed* (bounces baked, direct light and shadows live: the usual choice for the sun) or *Realtime*.

3. **Bake:** Window > Lighting > Generate Lighting. A baked lightmap costs about 1 ms on a 1080p frame.

4. **Moving objects** take their bounced light from a Probe Volume component (add one and bake again); metal and glass reflect a Reflection Probe's cubemap inside its box.

5. Auto Generate bakes again a second after you stop editing; the status bar says "Lighting out of date" until it does. The bake is saved next to the scene.

Lighting Updates: Manual (the default) keeps bounced light as last computed while you drag things; Update Lighting (Ctrl+Shift+L) refreshes it; Preview Lighting shows a rough bake half a second after you stop. **Realtime GI** (voxel global illumination [45]) shows bounced light while you're still arranging, with nothing to bake, and runs 4 to 8 times faster on a Vulkan GPU; set Render > Device to GPU Compute and Generate Lighting bakes on the GPU too.

Setting	What it does	When to change it
Lightmap Resolution	texels per metre (Unity's default is 40)	lower for big scenes, higher for sharp contact shadows
Indirect Samples, Bounces	rays per texel; how many times light bounces	raise if noisy; 3 or 4 for interiors
GI Resolution, Realtime Resolution	the live GI's share of the screen; realtime lightmaps' texels per metre	Half for a cleaner Scene view; higher for small rooms
Indirect Intensity	scales the baked bounce	artistic control
GI Device, Voxel Resolution	where realtime GI gathers; 64, 128 or 256 voxels across the scene	CPU only to compare; higher for small detail
Probe Min / Max Spacing	probe density near geometry / in open space	lower min spacing for small rooms

7.4 Camera filters

Camera filters are Unity's post-processing stack as components on a camera (Add Component > Camera Filters, then Add Filter): colour grading, bloom, vignette, chromatic aberration, lens distortion, film grain, CRT, pixelate, edge outline, sharpen, halftone, hatching, line art, paper, a colour ramp, and whole looks: **Retro Console** (PlayStation, N64, Saturn, DOS: wobbling vertices, affine textures, dithered colour [64]) and **Anime / Manga**. They show in the Game view, the Camera Preview, F12 (traced renders too) and camera sequences; the Scene view can look through them.

Example: a PS1-style render

1. Select the camera, Add Component > Camera Filters, Add Filter > Retro Console, Console: PlayStation.
2. F12: the wobbling vertices, affine textures and dithered colour show in the path-traced render too.

7.5 Developing a render: the Editing tab

Window > Editing (or the Editing layout) develops a render or an image the way Lightroom does: a stack of adjustments (Exposure, Contrast, Highlights / Shadows, white balance, a Tone Curve, HSL, Clarity, Dehaze, Sharpen, Noise Reduction, Vignette, Crop), each limited by a mask if you like (a gradient, a tone or colour range, a brush), applied in linear light and never written into the source: Save keeps the stack beside the image, Export writes the result, Use as Camera Look gives the camera the same look live, Apply to Texture puts an edit on a texture.

7.6 A scene opened from Unity

A Unity scene opened here looks as its Game view does: its fog mode, skybox, camera flags, post-processing (the ACES tonemapper, exposure, grading, bloom, vignette), the lights' shadows and cookies, its reflection probes, and Unity's own baked lightmaps

and light probes (Lighting > Use Unity's Lightmaps, read by your installed Unity in the background). The match is measured station by station against Unity's renders (`docs/hci/compare-unity-vrchat-2026-11.md`).

Where it comes from

The rasterizer follows EEVEE and the path tracer Cycles [71]; the lighting modes, lightmaps, probe volumes and post-processing follow Unity [122]; the physical sky is Hosek and Wilkie's [17]; realtime GI is Thiedemann et al.'s [45]; the microfacet model and its sampling are Cook–Torrance, Walter et al. and Heitz [7, 48, 14]; the denoiser is Dammertz et al.'s [9]; path guiding is Ruppert et al.'s [39]; the bloom is Jimenez's [22]; the theory is in [54] (ray tracing, Monte Carlo, tone reproduction) and [52] (lighting, post-processing). Learn: *How This Editor Renders*, *Lights and Materials*, *Render Engines*, *Light, Sky and Tone Mapping*, *Baked and Realtime Lighting*, *Rendering on the GPU*, *Editing: Develop a Render*.

Chapter 8

Rigging, skinning and animation

8.1 Skeletons and skins

Bones are GameObjects with a Bone component under a root with an Armature, as Unity imports a character. A mesh follows them through its Armature (Skin) modifier: each vertex moves with the bones its vertex groups are named after. Everything for characters is in the **Rigging** window: Skeleton, Skin, Transfer Weights, Vertex Groups, Weight Paint, Auto-Rig, IK, Spring Bones, Anime Face Normals. Select a bone and drag its blue tail handle to aim and stretch it; Ctrl+E extrudes a new bone from the tip; IK targets are boxes and poles diamonds, coloured by side (left blue, right red); FK Controls puts circles on bones to turn them by [90].

Automatic weights by inpainting (Epic's method [1]): Bind: Automatic Weights gives each vertex that is clearly nearest one bone, and that the bone can see, to that bone; the rest get the smoothest weights that agree with them, so nothing bleeds across a gap. Bind: Envelope is the quick distance-only binding. Transfer Weights dresses a character: drag the rigged body onto From; close matches copy the body's weights and everything else is inpainted. **Weight Paint** fixes weights by hand (Add, Subtract, Draw, Blur, a falloff, Mirror X; Ctrl or the pen's eraser subtracts).

IK. Add IK to Bone on a shin or a forearm makes a target and a pole; GameObject > Rig > Humanoid / Quadruped makes a whole skeleton with IK; Auto IK finds the limbs of an imported Mixamo, Unity or Rigify skeleton by name; an IK constraint's Weight 0 is FK, and Switch to FK / IK keeps the pose. **Auto-Rig with markers** (Mixamo's way [69]): select the mesh, Auto-Rig > Humanoid or Quadruped, click the markers the diagram asks for (chin, wrists, elbows, knees, groin; Symmetry mirrors), then Build Rig: the skeleton, IK on every limb and the inpainted weights.

Example: rig a character in five minutes

1. Import or model the character standing in a T- or A-pose, facing +Z (Unity's forward).
2. Select it; Rigging > Auto-Rig > Humanoid. Click the chin, the left wrist, the left elbow, the left knee and the groin.
3. Build Rig. Select a hand's IK target and move it: the arm follows, the mesh with it.
4. Fix any weight with Weight Paint (select the bone to paint its group).

Spring Bones make hair, ears, tails and cloth strips swing after the animation (Stiffness, Damping, Gravity, Radius and Collide against the body's capsules, Bend, Root To Tip, Separate, a Wind component), found automatically by name on imported characters (Mixamo, VRM, Japanese rigs); Bake writes the swing into a clip. **Soft tissue** on one bone trails the body's acceleration and keeps its volume.

8.2 The Animation window

Window > Animation is Unity's Animation window [122], a dopesheet laid out as Unity's: a Summary row of every key, an object row per animated object, the property rows under it with their axes, each row a key button and a value field you can type into.

1. Select the object (a character's root, a door) and press Create: it gets an Animation component and a clip, `Assets/Animations/<name>.anim`.
2. Turn on Record (Ctrl+Shift+K). Move the playhead along the ruler and pose things with the gizmo or the Inspector: every change becomes a key at the playhead; right-click any Inspector field > Insert Keyframe (or I over it) keys a light's intensity, a material slot's colour, a modifier's amount, a blend shape's weight.
3. Press play in the window (or turn on Preview and scrub); in Play (Ctrl+P) the Animation component plays the clip: Loop, Once, Ping Pong or Clamp Forever, at any speed.

Keys: click, Shift+click or a box to select; drag sideways to retime; Delete; Ctrl+C / Ctrl+V at the playhead; K keys every track; comma and full stop step through the keys; right-click for Auto (Unity's Clamped Auto), Linear or Constant; Add Property is a tree of the objects with their components. **Curves**: the window's Curves view shows how each value travels between keys; drag a key's tangent handles (Free Smooth), or right-click for Broken (a bounce), Flat (an ease) or Clamped Auto. A clip names objects by their path below the animated root, so it plays on any character with the same bone names; IK runs after the clip, so animating the IK targets is enough for arms and legs. Animation on 2s or 3s holds frames for the anime look.

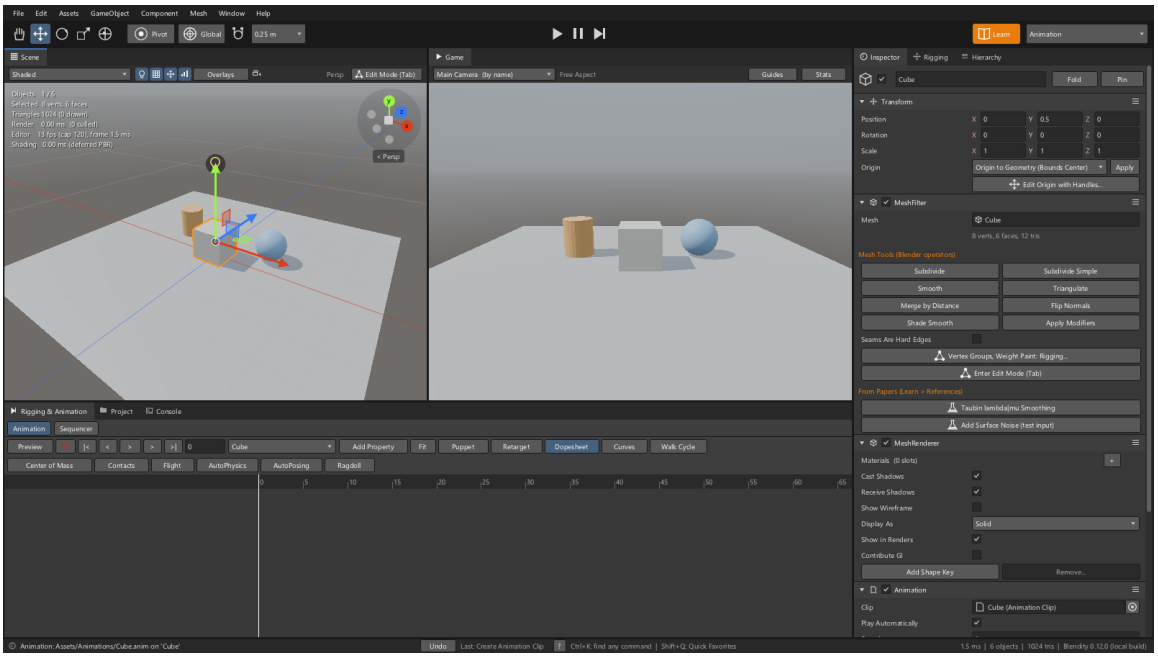


Figure 8.1: The Animation layout: the Animation window along the bottom with Preview, Record, the clip, Add Property, Puppet, Retarget, Dopesheet / Curves, Walk Cycle and the physics tools; the Rigging tab beside the Inspector.

Example: a ball that bounces

1. Key the ball's Position y at 2 on frame 0, 0 on frame 15, 2 on frame 30.
2. Curves view, click the Position.y track. Right-click the key on frame 15 > Broken.
3. Drag its left handle steeply down and its right handle steeply up: the ball now hits the floor hard and leaves fast. Right-click the top keys > Flat for a hang at the top.

8.3 Puppet Posing and physics for animation

Turn on AutoPosing and the selected character's hands, feet, head and hips get controls in the Scene view: drag one and the body follows, as motion capture would put a body there (a pose prior trained on the CMU database [34, 72]), the planted feet staying. Any humanoid rig works (Mixamo, the UE5 mannequin, Character Creator, Daz, VRoid, Biped, Rigify; T or A pose). The Puppet menu holds Reach, Motion Trails, Onion Skin, the Zoetrope, Motion Check (sliding feet, pops and shaking, each with a Fix), the Smooth Brush and Lock in Place. As in Cascadeur [79]: Center of Mass weighs the body from its skin and follows it through a clip, Contacts are found from the motion and pinned, Flight corrects a jump to a true parabola with its angular momentum kept, AutoPhysics makes the whole clip physically possible, take-offs and landings included, each shown as a ghost before it is applied; Ragdoll (Jolt) plays the clip with motors so hits and falls react. Walk Cycle makes walking and running cycles from the rig (feet never sliding, two legs or four), Footsteps walks a character along a curve on footprints you can move, Locomotion walks and runs it live in Play. AI Inbetweening (an optional download) fills the motion between two keys in a style, shown as a ghost. Every assistant is a switch in Preferences; hand animation never changes.

8.4 Retargeting

With a clip open on the rig it was made for, Retarget lists the scene's other humanoid rigs; pick one and the clip is remade for it by what each body part does (the 21 parts of the humanoid map found on any naming scheme), the hips' moves scaled to its height and an A pose's arm drop reconciled with a T pose as Unity's Mecanim reads them [114]; the new clip is saved beside the source's, set on the rig and opened. Fingers, the face and IK are not carried.

8.5 Sequencing clips

Window > Sequencer is a Premiere-style timeline for clips (instead of a state machine): press Create and drag `.anim` clips from the Project window onto its tracks; each bar is a clip and the object it plays on; drag to move, its ends to trim; bars overlapping on one track crossfade; a higher track wins; Triggers (a key, a clip, blend seconds) jump to a clip in Play. The Sequencer also holds storyboard cards and camera cuts for animatics, WAV sound mixed into Render Animation's video, footage, keyed opacity and transform, Add Edit (Alt+B) and Ripple Delete (Shift+Delete).

Where it comes from

Bones, skins and IK follow Blender's [71, 50]; the weights are Epic's inpainting [1]; the marker auto-rig is Mixamo's workflow [69]; the Animation window, Record, the tangent modes, Humanoid retargeting and `.anim` clips are Unity's [122, 114]; the physics for animation follows Cascadeur's ideas [79]; the pose prior follows ProtoRes [34] on CMU data [72]; the spring chains follow Jakobsen and the position-based solvers [21, 26]; the theory is in [52] (animation, physics). Learn: *Modifiers vs Components* (vertex groups, weights, bones, IK, keyframes, curves, sequences), *Play Mode and the Game Loop*, *Animating Any Property*.

Chapter 9

Files, other programs and Live Link

9.1 The project

The Project window is `Assets`: nested folders, drag to move, F2 to rename, Delete to the Recycle Bin or Trash; references to a moved or renamed material, texture or clip keep working. Scenes are text files (`.scene`); Ctrl+S saves, New or Open asks "Save changes?" first; the Console's `save`, `open` and `new` do the same.

9.2 Import

File > Import Model (or drop files on the window): OBJ + MTL, FBX, glTF, STL, PLY and USD text, with their materials and textures and, for rigged files, their skeletons, skins, bind poses, shape keys and animations (`Assets/Animations/<model>/`); MikuMikuDance `.pmx` / `.pmd` characters rigged, toon-shaded and dancing to `.vmd`; VRM avatars with their humanoid names, expressions, spring bones and MToon look; BVH motion capture; Collada; `.blend` through your installed Blender. The **Import Preview** shows the model first with its shading switchable (As Imported, PBR, or a toon style) and an Outline switch; Import writes it, Cancel nothing. The texture finder reads the folders around the model and pairs each bare material with the texture set its name is likeliest to own.

9.3 Export

File > Export: every mesh format Blender 5.2 exports (OBJ, FBX binary or ASCII, glTF `.glb` or `.gltf`, STL, PLY, USD) with Blender's options (Selected Only, Apply Modifiers, Scale, Up Axis, Triangulate, Keep Transforms). glTF and FBX carry a rig, its skin, its clip and the shape keys (visemes, blinks and expressions reach Unity, VRChat and Unreal); textures go with the file (procedural, painted and generated maps as PNGs, embedded in a `.glb`); Lightmap UVs adds a second UV set; Colliders and LOD Levels are named for Unity, Unreal or Godot; Bake Textures turns procedural materials into images; an export asked for while a remesh or a bind still works on what it writes waits for it. Every format was checked both ways against Blender 5.2.

9.4 Unity projects, VRChat, Godot, Blender, Unreal

Unity: File > Import Unity Package… unpacks a `.unitypackage` into `Assets/Imported/<package>/` with Unity's GUIDs kept; a prefab or a `.unity` scene comes in with its hierarchy, meshes, materials (converted by shader: Standard, URP, lilToon, Poiyomi, MToon, UTS2), lights, cameras, colliders, nested prefabs and overrides, and every other component as an Engine Component you can edit under Unity's names; File > Open (Unity Scene) opens a `.unity` as the scene with the look Unity gives it (chapter 7);

Unity `.anim` clips import and export; File > Export Unity Package… writes the work back under the same GUIDs. Nothing of Unity's is ever written over: an edit of a Unity clip or material saves to a Blendity copy beside it.

VRChat: the **Avatar** window (after the CATS plugin [68]) has Fix Model, Create Visemes, Model Options, Pose and Shape tools, Decimation, Bone Root and Bone Merging, Eye Tracking, the Texture Atlas, Merge Armatures, Attach Mesh, Translate and Export for VRChat, with a checklist against VRChat's requirements [130]; the Profiler's Budget ranks a scene or avatar against the PC and Quest performance ranks; an avatar's VRChat components (the Avatar Descriptor, PhysBones, colliders, contacts, constraints) come in as Blendity's and go back through the package or Live Link; a world's Scene Descriptor and Light Volumes too.

Godot: `.tscn` scenes come in with their nodes, primitives, materials, lights, cameras and collision shapes and go back with the objects' transforms, scripts kept [104]. **Blender:** `.blend` files come in through the installed Blender (`blender path` names it); the Blender bridge's Send to Blendity brings a scene over. **Unreal:** the bridge sends a level through its FBX exporter with each actor's place and Blueprint class.

9.5 Live Link

Live Link sends the scene to Blender, Unity, Godot or Unreal *as you work*: objects, transforms, meshes (a drag in Edit Mode streams while it happens), materials, textures by their pixels, lights, cameras, a rig's poses and skins, clips baked as the engine's own, colliders and LODs, the world and the render camera's look, VRChat's components where the SDK is there. It starts with Blendity (Preferences > Live Link) and writes where it listens, so a bridge connects by itself. To put a bridge in: Window > Live Link > Install a Bridge > Blender (found and enabled for you), or Unity / Godot / Unreal Project… and pick the project's folder (in Unity: Package Manager > Add package from disk > the bridge's `package.json`). The window lists who is connected. It is one way: edits on the other side are not sent back. Work on a copy of a project you care about.

Where it comes from

The formats follow Blender's importers and exporters [71] and their specifications [44, 59, 65]; Unity's files follow its manual [122, 119, 113]; VRChat's limits and components follow its creator documentation [130]; the Avatar window follows CATS [68]; Godot's format its documentation [104]. Learn: *Assets, Scenes and Files*, *Coordinate Systems and Handedness*.

Chapter 10

The Learn tab and the console

10.1 The Learn tab

F1 (or the orange Learn button) opens the Learn tab; the Learning layout puts it beside the Scene view. Each of its 45 lessons connects three layers: the Unity concept you can see on screen, the equivalent Blender concept with the folder in Blender's source where it lives, and the theory behind it, cited from the reference books [54, 53, 52] and the papers behind the features (the *References* lesson). Pick a lesson on the left; your progress is saved; Try it buttons drive the editor for you so you can watch the concept happen; click a question to reveal its answer; hover over almost any button or field in the editor to see its Blender equivalent; the ? beside the status bar's hint opens the lesson for what you are doing. Index C of this guide lists every lesson with its sources.

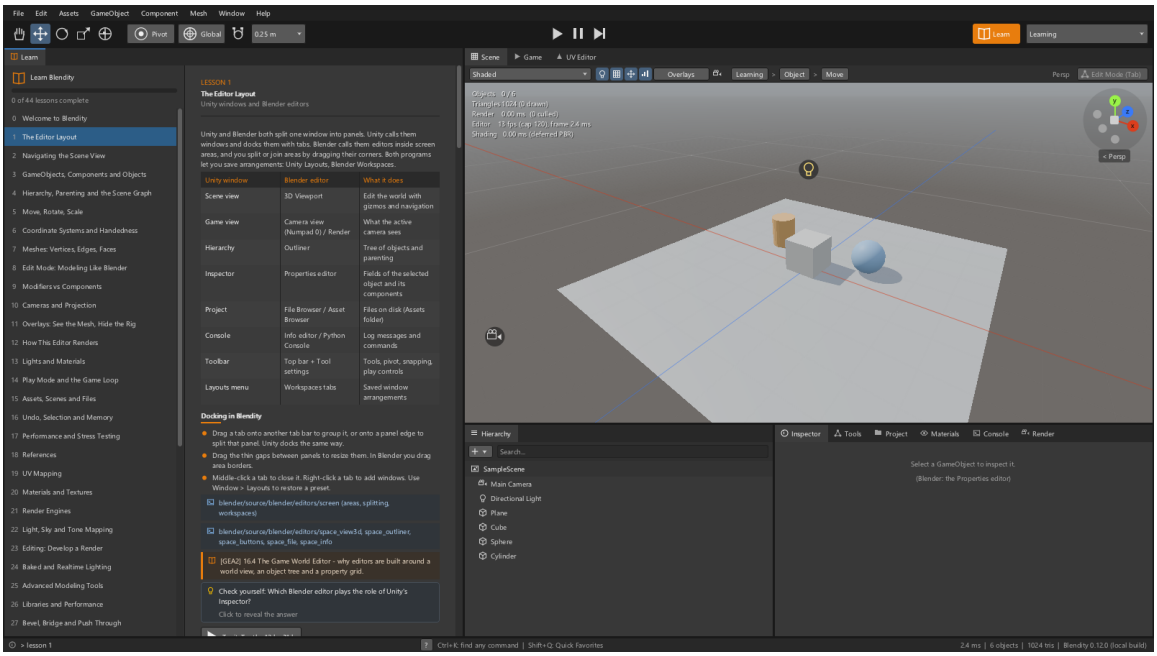


Figure 10.1: The Learning layout: the lesson list, a lesson with its table, references and Try-it buttons, and the Scene view the buttons drive.

The lessons, in order: Welcome to Blendity; The Editor Layout; Navigating the Scene View; GameObjects, Components and Objects; Hierarchy, Parenting and the Scene Graph; Move, Rotate, Scale; Coordinate Systems and Handedness; Meshes: Vertices, Edges, Faces; Edit Mode: Modeling Like Blender; Modifiers vs Components; Cameras and Projection; Overlays; How This Editor Renders; Lights and Materials; Play Mode and the Game Loop; Assets, Scenes and Files; Undo, Selection and Memory; Performance and Stress Testing; References; UV Mapping; Materials and Textures; Render Engines; Light, Sky and Tone Mapping; Editing: Develop a Render; Baked

and Realtime Lighting; Advanced Modeling Tools; Libraries and Performance; Bevel, Bridge and Push Through; Rendering on the GPU; Points, Lines, N-gons and Booleans; Drawing in Perspective; Customizing the Editor; Sculpting; Painting the Surface; The Hand-painted Look; Stylized Environments; Drawing like SketchBook; Papers; Blender's Shader Nodes in the Textures Window; Fog with a Volume Material; Working Faster with Textures and Materials; Flatten to the Camera; Shapes by the Camera's Angle; Animating Any Property; The Books and the References.

10.2 The console

Every tool can be driven from the Console's command line (type `help`; Up and Down recall what you typed; a mistyped command is answered with the nearest known ones), which is also how scripts and the tests drive the editor. None of it is needed any more: every verb an artist uses has a menu item, a button or a field. A few that are handy by hand:

Command	Does
`set Camera.FStop 1.4`	sets any field of the selected objects' components (enums by text)
`layout UV Unwrapping`, `alltools on`	layouts and Show All Tools
`skin bind`, `skin transfer Body Shirt`	automatic weights, weight transfer
`autorig humanoid`, `autorig build`	the marker auto-rig
`anim new`, `anim frame 30`, `anim record on`, `anim retarget <rig>`	the Animation window
`meshmaps 1024 16`, `surface new`, `paint on`, `smat apply Worn Steel`	the paint pipeline
`export textures unity Exports/Crate --assign`	the texture set packed for Unity, assigned to the material
`sketch on`, `sketchguide perspective 3`, `sketchcanvas ground`, `sketchbuild box`	the sketch pen, its guides and canvases, solids from the lines
`sculpt on`, `sculpt brush clay`, `sculpt dynamesh 128`, `sculpt remesh 2000`	Sculpt Mode and the remeshers
`render wait`, `saverender out.png`, `renderanim 1 120 24`	a render held till done, saved; an animation
`import <file>`, `export glb Exports/Hero.glb`, `export unitypackage <folder>`	files in and out
`livelink start`, `livelink status`	Live Link
`libs`, `jobs`, `budget pc`, `keymap table`	the build's libraries, the running jobs, the VRChat budget, every preset's keys
`update check`, `update history`, `update install <tag>`	the updater
`vr start`, `vr sketch on`, `vr paint on`	the VR View

10.3 Headless runs and scripts

`Blendity --headless-screenshot out.png --size 1600x900 --cmd "<command>" …--frames N` runs the commands with no window and writes a picture; `BLENDITY_PROJECT` and `BLENDITY_TRASH` point it at a scratch project, never your `Assets`. The figures in this guide, the tests and the Unity comparison harness are made that way.

10.4 Checking how it works for you

`docs/hci/checklist.md` lists 118 tasks across the editor, each with its steps, what should happen and a target time; its interactive page times each task with a stopwatch and keeps your results so the slow ones can be improved first. How the editor's ease of use was measured (the keystroke-level model [4], the Fitts audit [11], the walkthroughs [37], the heuristics [32]) is in `docs/hci/README.md`.

Chapter 11

Pen, fingers, the Draw tab, papers, the board and VR

11.1 Fingers and a pen on a 2-in-1

Tablet Mode (the tab menu, or Preferences > Pen and touch: Auto follows the laptop's fold) makes the targets half as big again (7 mm for a pen), and then fingers navigate while the pen works: one finger drags to orbit (a flick glides on), taps to select, double-taps to frame; two fingers pinch to zoom, drag to pan, twist to turn; one finger scrolls a list; the pen selects, moves, draws and paints, its side button held orbits (Shift pans, Ctrl zooms), its side button clicked, or the pen or a finger held still half a second, opens **the Wheel**: one radial menu with Undo, Delete, Extrude, the brushes, sizes and symmetry of whatever mode you are in; right-click any menu row > Add to the Wheel puts a command on it. A finger's press waits a moment before it counts, so a pinch never starts a box select; a palm resting on the screen while the pen is near is ignored. On Linux fingers and pens come through XInput 2; on a Mac the trackpad's pinch and twist navigate and a tablet's pen has its pressure and tilt. The pen's pressure drives the sculpt and paint brushes (a Pressure Curve in the brush settings), its tilt turns Flatten's and Scrape's plane and widens a drawing brush, its eraser end erases and inverts.

11.2 A pen display's dial and express keys

Pen displays such as XPPen's send their express keys as keyboard shortcuts and their dial as a key pair or the wheel; Blendity answers them where the pen draws: Ctrl - / Ctrl = zoom about the pen, [/] the brush size, 4 / 6 turn the canvas and 5 sets it upright (Krita's keys [76]), B brush, E eraser, Space held pans, Ctrl 0 fits; More > Wheel / Dial chooses what the wheel does.

11.3 The Draw tab

Window > Draw is the pad: the scene from the Scene or the Game camera, or blank Paper, the pen drawing in that perspective while the keyboard moves the view (the arrows, A / D, W / S orbit and dolly the camera while the pen is up), every stroke placed in the scene where it was drawn. It works like SketchBook Pro's canvas [109]: one slim toolbar (Undo, Redo, Brush, the color with its opacity, Size, Ruler, Ellipse, Curve, Symmetry, Perspective, Steady Stroke, Predictive Stroke, Layers, the zoom and More), the pad zoomed, panned, turned and flipped with the wheel, a middle-drag or Space+drag and two fingers, Tab to hide the tools. The brush palette holds Pencil, Pen, Inker, Felt Pen, Marker, Airbrush and two erasers, with the puck (drag sideways for the size, up and down for the opacity; B held and dragged puts it at the pen) and Brush Properties on a double-click. The **Layers** window composites the sketch's layers

as Photoshop does [86]: opacity, the 27 blend modes, groups, locks, Clip to the layer below, a layer mask from a selection; More > Select has rectangle and lasso selections (Alt+Backspace fills, Delete cuts, Ctrl+D deselects) and Ctrl+T Free Transform; More > Shapes has Rectangle, Ellipse, Polygon, Star and Illustrator's Pen with fills, artboards and SVG in and out. Place in 3D puts each stroke in the scene at the depth under the pen; Build and Lift make geometry from the construction lines (chapter 4).

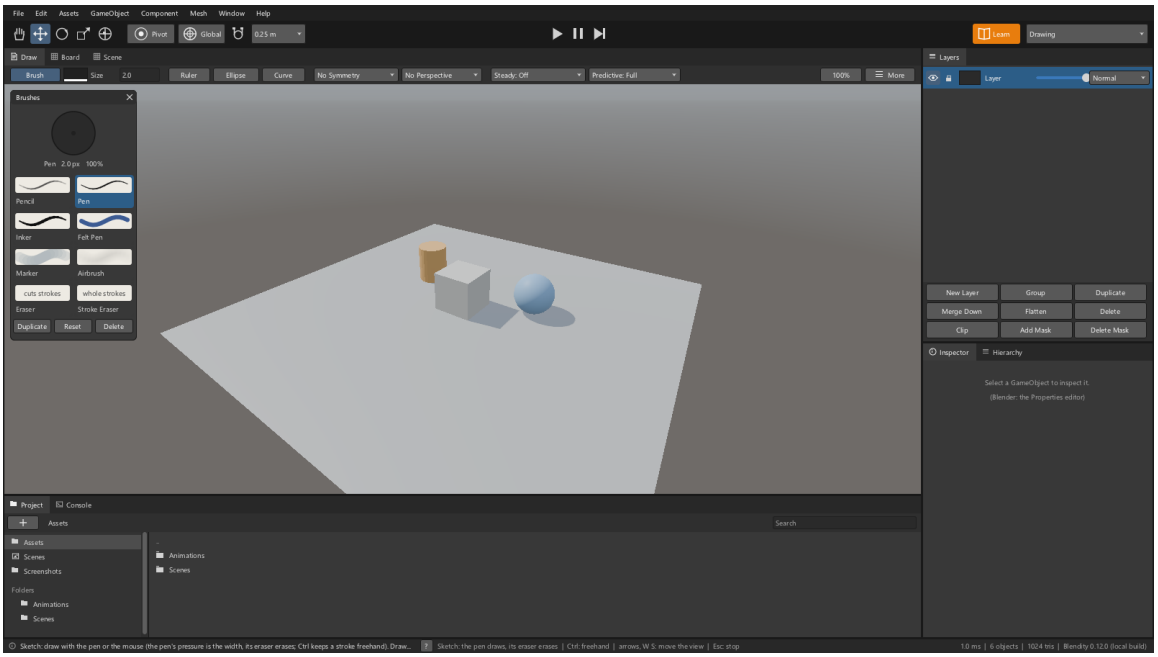


Figure 11.1: The Drawing layout: the Draw tab's pad with its toolbar, the Layers window and the Project window.

11.4 Papers

More > Source > Paper draws on a paper: a page of its own in `Assets/Papers`, not in the scene. New Paper, Open Paper, Duplicate and Delete live under More > Paper; a paper saves itself a second after each change and on closing, and the one used last opens again. A paper can be pinned to the board, where double-clicking it draws on it.

11.5 The board

Window > Board is a PureRef-style reference board [74]: drop images from the system, drag them from the Project window, Add Image... or Ctrl+V a screenshot; the wheel zooms about the pointer, a middle-drag pans, Home fits everything, F frames the selection; drag to move, a corner to scale, the round handle to turn (Ctrl: 15-degree steps); Ctrl+] / Ctrl+[to the front and back, Ctrl+D duplicates, Delete removes; its own undo history; it saves itself to `Assets/Boards` and opens again next time; Boards > Export... writes PNG, JPEG or PDF.

11.6 VR View

Edit > VR View (`vr start`) shows the scene in an OpenXR headset (SteamVR, Quest Link, Windows Mixed Reality [83]) at life size, standing on the ground below where the Scene view looks. Squeeze a grip and drag to move the world; both grips to scale or turn it; hold the right trigger to throw a teleport arc; the left stick walks, the right snap-turns; A shows a tape measure. `vr sketch on`: the right trigger draws ribbons or tubes in the air with pressure and a palette, into a Stroke object that renders and exports like any mesh. `vr paint on`: paint the object with the desk's brush where the controller's ray meets it. It needs the build with Blender's libraries, on Windows.

Learn

The Editor Layout > Fingers and a pen on a 2-in-1, *Customizing the Editor*, *Drawing like SketchBook*, *Papers*, *Drawing in Perspective*.

Chapter 12

Performance, the minimum machine and troubleshooting

12.1 The minimum machine and the quality tiers

A thin laptop such as the HP Envy 13.3-inch 2-in-1 (an Intel i7 with 4 cores / 8 threads, 8 GB of RAM shared with Iris Xe graphics, a 1920×1080 touch screen at 150 %) runs Blendity on the Low tier and says so on the first run: editing frames at about 35–40 fps, orbiting at 60. A desktop with a 4-core CPU and a GTX 1050 Ti is faster. The GPU is for path tracing, baking and realtime GI; a ray-tracing card (RTX, RX 6000+, Arc) is recommended for them.

Tier	Views drawn at	Shadow map	Frame cap	For
Low	67 % (scaled up)	1024	60 fps	the minimum machine
Medium	85 %	2048	120 fps	a mid-range PC
High	100 %	the scene's	144 fps	a fast PC

Renders (F12) always use the scene's own settings. CPU Threads limits how many cores Blendity uses; the **Profiler** window shows where a frame's time goes and its Budget ranks the scene against VRChat's limits. Tips for a scene that stays smooth: bake lighting, keep Lighting Updates on Manual, keep Subdivision Surface's Levels at 1 while modelling, turn on the Low tier for heavy scenes, keep the Game view and the Camera Preview closed while you don't need them (they keep their last picture during a drag anyway).

12.2 When something refuses

A refusal is a toast over the Scene view, and most carry a Fix:

- "Edit Mode needs an active object with a MeshFilter": select a mesh first.
- "Paint needs a surface / UVs": Fix unwraps or makes the surface.
- "… can't run during Play": Fix stops Play.
- "… while a job runs on it": the tray's Cancel, or wait; an export waits by itself.
- "Generate Lighting: no object contributes GI": Mark Static Meshes.
- A dim button: its tooltip says why (no selection, no libraries, the wrong mode).
- "the Scene view has no Rendered mode": path trace with F12; the Render window's Preview is the quick look.

12.3 Without Blender’s libraries

Preferences > Libraries shows why a build runs without them (missing, damaged, turned off, offline) and offers Download or the switch. Without them: no Boolean or Decimate modifier, no EXR, no OpenColorIO views, slower path tracing and the À-Trous denoiser, sphere physics in Play, and no MP4 of its own (Render Animation offers the libraries or an `ffmpeg` on the PATH). Everything else works the same.

12.4 The GPU

A CPU badge in the Scene view’s bar says the views draw on the CPU although Auto wanted the graphics card; its tooltip says why (no Vulkan, a material or world the card doesn’t draw yet: volumes and the Principled extras go to the CPU and say so). Render > Device picks the path tracer’s device; `gpu list` names the cards; a small GPU gets a quarter of the RAM and is never overrun.

12.5 Files and projects

Your project is a folder with `Assets` inside; copy it whole. A scene names its materials, textures and clips by project path and keeps values as a fallback, so a scene opened without its assets still shows. The Project window’s moves and deletes keep references working; deletes go to the system’s trash. Never edit a Unity project you care about in place: import a copy, or let Live Link write into a copy.

12.6 Where to ask

Open an issue on GitHub (`https://github.com/EmilianoManaloIV/blendity/issues`) with the Console’s log (Copy in the Report dialog, or `BlendityProject/Screenshots` for a picture). If something in the documentation is out of date, that is a bug too.

Learn

Performance and Stress Testing > Quality tiers, *Rendering on the GPU > A small graphics card*, *Libraries and Performance*.

Chapter 13

Licensing and credit

Blendity follows Blender's design closely and is licensed to be compatible with it, with the libraries Blender bundles, and with the way it is written: by an AI under a person's direction. This chapter states the licence terms once, the reasoning behind them, and who is credited for what.

13.1 The terms

The licence, in four lines (ADR 0002, ADR 0104)

- **The source** is GPL-2.0-or-later [60], the same as Blender's [71]. Every source file carries `SPDX-License-Identifier: GPL-2.0-or-later` [67]; the full text is the root `LICENSE`.
- **The executables** are distributed under GPL-3.0-or-later [61], because they combine that source with Apache-2.0 code [66] (MikkTSpace, the Cycles sky evaluation, and when built with Blender's libraries Embree, OpenImageDenoise, oneTBB, Manifold, Open PGL and others), and Apache-2.0 combines with GPL version 3 but not with version 2 alone [62]. This is Blender's own situation.
- **Your work is yours.** The GPL covers the program, not its output: scenes, meshes, textures, renders, animations and exported files belong to whoever made them.
- **Third-party notices** ship with every binary (`licenses/`), and any new copied code or library adds its row to `licenses/README.md` in the same change.

13.2 Why these terms are compatible with Blender's

Blender is GPL-2.0-or-later. A program that borrows its design and ports pieces of its source must be under a licence the GPL accepts, and "GPL-2.0-or-later" for the source keeps the two interchangeable in both directions: code could move from Blender's tree into Blendity's and back without a relicensing. The "or later" clause is what lets the binaries be GPL-3.0-or-later: Blender's own release binaries bundle the same Apache-2.0 libraries and are distributed on the same basis.

Three kinds of third-party material are in or near the tree, each handled differently:

- **Code copied unmodified from Blender's tree** into `extern/`: ufbx and fast_float (MIT), MikkTSpace (Apache-2.0), the Hosek–Wilkie sky (BSD-3-Clause), QuadriFlow with LEMON and pcg32 (BSD-3-Clause, Boost, Apache-2.0), cuew (a modified Apache-2.0), `libc_compat`. Their licence texts are in `licenses/` and `extern/README.md` says where each came from.

- **Code ported, not copied**: `src/render/sky.cpp` re-implements the sky radiance evaluation of Cycles' `intern/cycles/kernel/svm/sky.h` (Apache-2.0, 2011–2022 Blender Foundation) in Blendity's own functions. The file's header keeps that notice word for word, its SPDX line names both licences (`GPL-2.0-or-later AND Apache-2.0`), and the file as a whole is distributed under GPL-3.0-or-later through the "or later" clause (ADR 0002, amended by ADR 0104).
- **Code read, not copied**: lilToon's lighting formulas [77] are re-implemented from reading its shader (MIT; ADR 0100); the Unity material converter reads Poiyomi's [81] and MToon's [84] property names; the Avatar window follows the CATS plugin's tools [68]; the physics for animation follows Cascadeur's ideas [79]. Nothing of their code is in the tree; they are credited in `CREDITS.md` and listed in `licenses/README.md` under "read, not copied".

Optional downloads are never part of a build: Blender's prebuilt libraries (fetched by the installer or by Blendity itself, each under its own licence, all GPL-3-compatible), ONNX Runtime [78] and the inbetweening network trained on the CMU motion capture database [72], and MoonRay [73] run as a separate program (ADR 0041). The Live Link bridges (`bridges/`) are Blendity's own code under the same GPL-2.0-or-later and run inside Blender, Unity, Godot and Unreal as those programs' plug-in terms allow; they link no code of Blendity's into them.

13.3 Compatible with an AI-assisted project

The code is written with Anthropic's Claude through Claude Code [70] from task specifications and decision records a person approves, and every change is reviewed by a person before it lands (`docs/AI_DISCLOSURE.md`). Three things make that compatible with the GPL and with reuse by others:

1. **The outputs are the user's.** Under Anthropic's terms [58] the model's outputs belong to the user who directed them, and the project licenses them to the extent copyright subsists in them: the human selection, direction, review and editing (a purely machine-written passage may carry no copyright of its own, and the licence claims none it does not have). Copyright is held by Emiliano Manalo and contributors; the `Co-Authored-By` trailer names the model as attribution, not as a rights holder.
2. **Nothing is claimed that isn't the project's.** Where a method comes from a paper, the paper is cited in the code's comments, in the Learn tab and in `research/FEATURES.md`; where code is copied, its licence rides along; where a program's design is followed, it is credited. A reader can tell, for every piece, what was borrowed and under what terms.
3. **The AI is a tool at build time, not in the product.** The editor runs no model except the optional, local AI inbetweening download; there is no telemetry and no cloud, and the one request it makes on its own is the optional daily check of GitHub's releases. A user of the GPL program depends on no service.

13.4 Due credit

Blendity stands on the work of others, and `CREDITS.md` names them: the Blender Foundation and the Blender authors, whose design and source every subsystem

follows [71]; Unity Technologies, whose editor the windows, shortcuts, Play mode, Animation window and asset formats follow [122]; Epic Games (Unreal: the keymap preset, the editor feel, FBX collision naming, Sequencer) [101]; the Godot Engine contributors [104]; Autodesk (Maya and 3ds Max presets) [89, 88]; Trimble (SketchUp's Push/Pull, drawing and inference) [110]; Adobe (Photoshop's brush keys and blend modes, Substance Painter's mesh maps and layer stacks, Mixamo's auto-rig) [86, 87, 69]; UModeler and Plasticity [111, 108]; Maxon's ZBrush and Pixologic's Sculptris [106, 107]; SketchBook [109], PureRef [74] and Krita [76]; VRChat [130], lilToon [77], Poiyomi [81], the VRM Consortium [84], VRoid [80], the MMD community [85] and the CATS plugin [68]; Nekki's Cascadeur [79]; DreamWorks' MoonRay [73]; REDSIM's VRC Light Volumes [82]; Carnegie Mellon's motion capture database [72]; Marschner and Shirley [54] and Gregory [53, 52], whose books the Learn tab teaches from; and the authors of every paper in the References chapter. Ideas are borrowed with their names attached; code only with its licence.

References

Everything used in Blendity, by kind: the papers whose methods became features, the books the Learn tab teaches from, the standards and licences, the libraries compiled in or linked, and the manuals of the programs Blendity follows. Each entry ends with the pages that cite it. The list is generated from the sources that cite things (`src/editor/lessons.cpp`, `research/FEATURES.md`, `licenses/README.md`) and a hand-written list (`docs/book/refs_extra.bib`); `tools/make_books.py --check` fails when an entry is never cited or a citation has no entry, so the list and the books cannot drift apart.

Papers, theses and talks

[1] R. Abdrashitov, K. Raichstat, J. Monteiro, C. Stoll, M. Dvoroznak, and A. Jacobson. “Robust Skin Weights Transfer via Weight Inpainting”. In: *SIGGRAPH Asia 2023 Technical Communications*. Cited by Learn: Modifiers vs Components. 2023 (cited on pp. 27, 30, 56).

[2] J. Amanatides and A. Woo. “A fast voxel traversal algorithm for ray tracing”. In: *Eurographics ’87*. Cited by Learn: References. 1987 (cited on p. 57).

[3] W. Baxter, J. Wendt, and M. C. Lin. “IMPaSTo: A Realistic, Interactive Model for Paint.” In: *NPAR 2004*. Cited by research/FEATURES.md. 2004 (cited on pp. 21, 22, 60).

[4] S. K. Card, T. P. Moran, and A. Newell. “The Keystroke-Level Model for User Performance Time with Interactive Systems.” In: *CACM 23(7)* (1980). Cited by research/FEATURES.md. (cited on p. 36).

[5] E. Catmull and J. Clark. “Recursively generated B-spline surfaces on arbitrary topological meshes.” In: *Computer-Aided Design 10(6)* (1978). Cited by Learn: Modifiers vs Components, Learn: References, research/FEATURES.md. (cited on pp. 17, 56, 57).

[6] T. Chen, Z. Zheng, D. Cohen-Or, and N. J. Mitra. “3-Sweep: Extracting Editable Objects from a Single Photo.” In: *SIGGRAPH Asia 2013*. Cited by research/FEATURES.md. 2013 (cited on pp. 14, 60).

[7] R. Cook and K. Torrance. “A reflectance model for computer graphics”. In: *ACM Transactions on Graphics 1(1)* (1982). Cited by Learn: References. (cited on pp. 26, 57).

[8] R. Cranley and T. Patterson. “Randomization of number theoretic methods for multiple integration”. In: *SIAM Journal on Numerical Analysis 13(6)* (1976). Cited by Learn: References. (cited on p. 57).

[9] H. Dammertz, D. Sewtz, J. Hanika, and H. Lensch. “Edge-avoiding À-Trous wavelet transform for fast global illumination filtering”. In: *HPG ’10*. Cited by Learn: References. 2010 (cited on pp. 26, 57).

[10] T. Duff et al. “Building an orthonormal basis, revisited”. In: *Journal of Computer Graphics Techniques 6(1)* (2017). Cited by Learn: References. (cited on p. 57).

[11] P. M. Fitts. “The information capacity of the human motor system in controlling the amplitude of movement.” In: *J. Exp. Psychol. 47(6)* (1954). Cited by research/FEATURES.md. (cited on p. 36).

[12] S. F. F. Gibson. “Constrained Elastic Surface Nets: Generating Smooth Surfaces from Binary Segmented Data.” In: *MICCAI ’98*. Cited by research/FEATURES.md. 1998 (cited on pp. 17, 60).

[13] Y. Gryaditskaya, F. Hähnlein, C. Liu, A. Sheffer, and A. Bousseau. “Lifting Freehand Concept Sketches into 3D.” In: *SIGGRAPH Asia 2020; Hähnlein, F., Gryaditskaya, Y., Sheffer, A. & Bousseau, A. (2022). *Symmetry-driven 3D Reconstruction from Concept Sketches.* SIGGRAPH 2022*. Cited by research/FEATURES.md. 2020 (cited on pp. 14, 60).

[14] E. Heitz. “Sampling the GGX distribution of visible normals”. In: *Journal of Computer Graphics Techniques 7(4)* (2018). Cited by Learn: References. (cited on pp. 26, 57).

[15] E. Heitz, S. Hill, and M. McGuire. “Combining analytic direct illumination and stochastic shadows”. In: *I3D ’18*. Cited by Learn: References. 2018 (cited on p. 57).

[16] S. Hill. “ACES Filmic Tone Mapping Curve (an analytic fit of the RRT + ODT)”. In: *Published fit, used by Unity’s Post Processing Stack*. The “ACES (full fit, Unity’s)” view transform of round 405 is this curve without the fit’s matrices. 2016 (cited on p. 23).

[17] L. Hosek and A. Wilkie. “An analytic model for full spectral sky-dome radiance”. In: *SIGGRAPH ’12*. Cited by Learn: Light, Sky and Tone Mapping, Learn: References. 2012 (cited on pp. 23, 26, 57, 58).

[18] J. Huang, Y. Zhou, M. Niessner, J. R. Shewchuk, and L. J. Guibas. “QuadriFlow: A Scalable and Robust Method for Quadrangulation.” In: *SGP 2018*. Cited by research/FEATURES.md. 2018 (cited on pp. 17, 60).

[19] T. Igarashi, S. Matsuoka, S. Kawachiya, and H. Tanaka. “Interactive Beautification: A Technique for Rapid Geometric Design.” In: *UIST ’97*. Cited by research/FEATURES.md. 1997 (cited on pp. 14, 60).

[20] A. Jacobson, L. Kavan, and O. Sorkine-Hornung. “Robust Inside-Outside Segmentation using Generalized Winding Numbers.” In: *SIGGRAPH 2013*. Cited by research/FEATURES.md. 2013 (cited on pp. 17, 60).

[21] T. Jakobsen. “Advanced Character Physics”. In: *Game Developers Conference*. The Verlet / constraint-projection scheme of the spring bone chains (ADR 0049). 2001 (cited on p. 30).

[22] J. Jimenez. “Next Generation Post Processing in Call of Duty: Advanced Warfare”. In: *SIGGRAPH 2014 course: Advances in Real-Time Rendering in Games*. The bloom’s downsample and tent-filter upsample chain (render/camera_filter.cpp). 2014 (cited on p. 26).

[23] B. Kang et al. “Design of advanced color temperature control system for HDTV applications”. In: *Journal of the Korean Physical Society 41(6)* (2002). Cited by Learn: References. (cited on pp. 23, 57).

[24] B. Karis. “Real shading in Unreal Engine 4”. In: *SIGGRAPH course: Physically Based Shading in Theory and Practice*. Cited by Learn: References. 2013 (cited on p. 57).

[25] B. Lévy, S. Petitjean, N. Ray, and J. Maillot. “Least squares conformal maps for automatic texture atlas generation”. In: *SIGGRAPH ’02*. Cited by Learn: References, Learn: UV Mapping. 2002 (cited on pp. 18, 57, 58).

[26] M. Macklin, M. Müller, and N. Chentanez. “XPBD: Position-Based Simulation of Compliant Constrained Dynamics”. In: *Motion in Games*. Substepping and compliance in the strand and spring solvers (ADR 0049; the XPBD rods of task 0168). 2016 (cited on p. 30).

[27] M. Meyer, M. Desbrun, P. Schroeder, and A. Barr. “Discrete Differential-Geometry Operators for Triangulated 2-Manifolds (the cotangent Laplacian and Voronoi areas)”. In: Cited by Learn: Modifiers vs Components. 2003 (cited on p. 56).

[28] M. Mikkelsen. “Simulation of wrinkled surfaces revisited”. Cited by Learn: References. Master’s thesis. Master’s thesis, University of Copenhagen, 2008 (cited on p. 57).

[29] T. Möller and B. Trumbore. “Fast, minimum storage ray-triangle intersection”. In: *Journal of Graphics Tools 2(1)* (1997). Cited by Learn: References. (cited on p. 57).

[30] J. Motomura. “GuiltyGearXrd’s Art Style: The X Factor Between 2D and 3D”. In: *Game Developers Conference*. Toon shading’s painted inputs, custom normals and the per-vertex outline width (ADR 0048); Camera Flatten (ADR 0093). 2015 (cited on pp. 22, 61).

[31] M. Newell. “The utilization of procedure models in digital image synthesis”. Cited by Learn: References. PhD thesis. PhD thesis, University of Utah, 1975 (cited on pp. 13, 57).

[32] J. Nielsen. “Enhancing the explanatory power of usability heuristics.” In: *CHI ’94 (the ten heuristics)*. Cited by research/FEATURES.md. 1994 (cited on pp. 36, 62).

[33] M. Okabe and K. Ito. “Color Universal Design (CUD): How to make figures and presentations that are friendly to colorblind people.” In: Cited by research/FEATURES.md. 2008 (cited on p. 17).

[34] B. N. Oreshkin, F. Bocquelet, F. G. Harvey, B. Raitt, and D. Laflamme. “ProtoRes: Proto-Residual Network for Pose Authoring via Learned Inverse Kinematics.” In: *ICLR*. Cited by research/FEATURES.md. 2022 (cited on pp. 29, 30).

[35] K. Perlin. “Improving noise”. In: *SIGGRAPH ’02*. Cited by Learn: References. 2002 (cited on pp. 19, 57, 61).

[36] B. T. Phong. “Illumination for Computer Generated Pictures”. In: *Communications of the ACM 18(6)*. The Blinn-Phong preview spheres of the Materials window. 1975 (cited on p. 19).

[37] P. G. Polson, C. Lewis, J. Rieman, and C. Wharton. “Cognitive walkthroughs: a method for theory-based evaluation of user interfaces.” In: *IJMMS 36(5)* (1992). Cited by research/FEATURES.md. (cited on p. 36).

[38] R. Ramamoorthi and P. Hanrahan. “An efficient representation for irradiance environment maps”. In: *SIGGRAPH ’01*. Cited by Learn: References. 2001 (cited on p. 57).

[39] L. Ruppert, S. Herholz, and H. Lensch. “Robust fitting of parallax-aware mixtures for path guiding”. In: *ACM Transactions on Graphics 39(4), SIGGRAPH ’20* (2020). Cited by Learn: References. (cited on pp. 26, 57).

[40] C. Schlick. “An inexpensive BRDF model for physically-based rendering”. In: *Computer Graphics Forum 13(3)* (1994). Cited by Learn: References. (cited on p. 57).

[41] Sheffer, Levy, Mogilnitsky, and Bogomyakov. “ABF++: Fast and Robust Angle Based Flattening”. In: *ACM TOG*. Cited by Learn: UV Mapping. 2005 (cited on pp. 18, 58).

[42] P. Shirley and K. Chiu. “A low distortion map between disk and square”. In: *Journal of Graphics Tools 2(3)* (1997). Cited by Learn: References. (cited on p. 57).

[43] G. Taubin. “A signal processing approach to fair surface design.” In: *SIGGRAPH ’95*. Cited by Learn: References, research/FEATURES.md. 1995 (cited on pp. 17, 57).

[44] The Khronos Group. *glTF 2.0 Specification*. Cited by research/FEATURES.md. , Skins and Animations, 2021 (cited on p. 32).

[45] S. Thiedemann, N. Henrich, T. Grosch, and S. Müller. “Voxel-based Global Illumination.” In: *I3D ’11*. Cited by Learn: Baked and Realtime Lighting, Learn: References, research/FEATURES.md. 2011 (cited on pp. 24, 26, 57, 58).

[46] W3C. *Web Content Accessibility Guidelines 2.1*. Cited by research/FEATURES.md. , 1.4.3 Contrast (Minimum) and 1.4.11 Non-text Contrast, 2018 (cited on pp. 8, 62).

[47] I. Wald. “On fast construction of SAH-based bounding volume hierarchies”. In: *IEEE Symposium on Interactive Ray Tracing*. Cited by Learn: References. 2007 (cited on p. 57).

[48] B. Walter, S. Marschner, H. Li, and K. Torrance. “Microfacet models for refraction through rough surfaces”. In: *EGSR ’07*. Cited by Learn: References. 2007 (cited on pp. 26, 57).

[49] Wang et al. “PartUV: Part-Based UV Unwrapping of 3D Meshes”. In: *SIGGRAPH Asia*. Cited by Learn: UV Mapping. 2025 (cited on pp. 18, 58).

[50] C. Welman. “Inverse Kinematics and Geometric Constraints for Articulated Figure Manipulation”. Cited by Learn: Modifiers vs Components. Master’s thesis. MSc thesis, Simon Fraser University (CCD), 1993 (cited on pp. 30, 56).

Books

[51] J. M. Carroll. *The Nurnberg Funnel: Designing Minimalist Instruction for Practical Computer Skill*. Minimalist instruction: the user guide’s task-first chapters, short reading and error-recovery pages. MIT Press, 1990 (cited on p. 62).

[52] J. Gregory. *Game Engine Architecture. Volume II: Graphics, Motion, and Sound*. 3rd ed. The Learn tab’s [GEA2]: the rendering engine, lighting, post-processing, physics, game objects, the world editor. CRC Press, 2018 (cited on pp. 22, 26, 30, 33, 44, 55–61).

[53] J. Gregory. *Game Engine Architecture. Volume I: Foundations and Core Engine Systems*. 3rd ed. The Learn tab’s [GEA1]: tools and the asset pipeline, profiling, SIMD, parallelism, math, memory, the game loop. CRC Press, 2018 (cited on pp. 33, 44, 55–57, 59–61).

[54] S. Marschner and P. Shirley. *Fundamentals of Computer Graphics*. 5th ed. The Learn tab's [FoCG]: math, viewing, rasterization, meshes, texture mapping, ray tracing, colour. CRC Press, 2021 (cited on pp. 18, 22, 26, 33, 44, 55–61).

[55] J. Nielsen. *Usability Engineering*. The heuristic evaluation method behind the HCI passes (ADR 0085). Academic Press, 1993 (cited on p. 62).

Standards, specifications and licences

[56] Academy of Motion Picture Arts and Sciences. *Academy Color Encoding System (ACES)*. The RRT + ODT that the ACES view transforms approximate. AMPAS, 2026. URL: https://acescentral.com/ (cited on p. 58).

[57] American Society of Cinematographers. *ASC Color Decision List (CDL) Transfer Functions and Interchange Syntax*. The lift / gain / gamma order of the Color Grading effect. ASC Technology Committee, 2009 (cited on p. 58).

[58] Anthropic. *Anthropic Consumer Terms of Service and Commercial Terms*. The terms under which the model's outputs belong to the user who directed them (the authorship statement). Anthropic, 2026. URL: https://www.anthropic.com/legal/consumer-terms (cited on pp. 43, 62).

[59] Blender Foundation. *FBX binary file format specification (the io_scene_fbx exporter's encode_bin.py)*. The record tree, header, null records and footer the FBX writer follows. Blender Foundation, 2026. URL: https://code.blender.org/2013/08/fbx-binary-file-format-specification/ (cited on p. 32).

[60] Free Software Foundation. *GNU General Public License, version 2*. The licence of Blendity's source (GPL-2.0-or-later), as Blender's. Free Software Foundation, 1991. URL: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html (cited on pp. 42, 62).

[61] Free Software Foundation. *GNU General Public License, version 3*. The licence of the executables, which combine GPL-2.0-or-later code with Apache-2.0 libraries. Free Software Foundation, 2007. URL: https://www.gnu.org/licenses/gpl-3.0.html (cited on pp. 42, 62).

[62] Free Software Foundation. *Frequently Asked Questions about the GNU Licenses*. Why Apache-2.0 code combines with GPL-3 and not GPL-2-only (the "or later" clause). Free Software Foundation, 2026. URL: https://www.gnu.org/licenses/gpl-faq.html (cited on p. 42).

[63] ISO/TC 171. *ISO 14289-2:2024, Document management applications - PDF/UA-2: Accessibility of PDF*. The accessible-PDF profile the books aim at: tags, language, alt text, a logical reading order. International Organization for Standardization, 2024. URL: https://www.iso.org/standard/83216.html (cited on p. 62).

[64] M. Korth. *PSX-SPX: PlayStation Specifications (GPU Rendering, Dither Matrix)*. The Retro Console filter's PS1 look: integer vertices, affine texture mapping, the 4 x 4 dither matrix. no$psx, 2024. URL: https://psx-spx.consoledev.net/ (cited on p. 25).

[65] Pixar Animation Studios. *Universal Scene Description (USD) Documentation*. The USDA text form Blendity reads and writes. Pixar, 2026. URL: https://openusd.org/ (cited on p. 32).

[66] The Apache Software Foundation. *Apache License, Version 2.0*. The licence of Embree, OpenImageDenoise, oneTBB, MikkTSpace and the Cycles sky code ported in src/render/sky.cpp. The Apache Software Foundation, 2004. URL: https://www.apache.org/licenses/LICENSE-2.0 (cited on pp. 42, 62).

[67] The Linux Foundation. *SPDX License List and Identifiers*. The 'SPDX-License-Identifier' headers in every source file. SPDX Workgroup, 2024. URL: https://spdx.org/licenses/ (cited on p. 42).

Software and libraries

[68] absolute-quantum. *CATS Blender Plugin*. MIT. Fix Model, Create Visemes, Decimation, Eye Tracking, Merge Armatures: the Avatar window follows its tools (rounds 356-361). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on pp. 32, 43, 44).

[69] Adobe. *Adobe Mixamo*. The marker auto-rig's workflow (chin, wrists, elbows, knees, groin) and the bone naming the humanoid map knows. 2026. URL: https://www.mixamo.com/ (cited on pp. 27, 30, 44).

[70] Anthropic. *Claude Code*. The agentic development environment Blendity is written with, under human direction and review (docs/AI_DISCLOSURE.md). 2026. URL: https://claude.com/claude-code (cited on pp. 43, 62).

[71] Blender Foundation and the Blender authors. *Blender*. Version 5.2. GPL-2.0-or-later. The design and source Blendity's meshes, Edit Mode tools, UVs, materials, renderers, skins and IK follow; the Learn lessons name the file each piece mirrors. 2026. URL: https://projects.blender.org/blender/blender (cited on pp. 18, 22, 26, 30, 32, 42, 44, 55–62).

[72] Carnegie Mellon University. *CMU Graphics Lab Motion Capture Database*. The data the AI inbetweening network and Puppet Posing's pose prior were trained on: "The data used in this project was obtained from mocap.cs.cmu.edu. The database was created with funding from NSF EIA-0196217." 2026. URL: http://mocap.cs.cmu.edu/ (cited on pp. 29, 30, 43, 44).

[73] DreamWorks Animation. *OpenMoonRay*. Apache-2.0. An optional external renderer run as a program of its own (ADR 0041, moonray.yml); not linked, not shipped. 2026. URL: https://github.com/dreamworksanimation/openmoonray (cited on pp. 23, 43, 44).

[74] Idyllic Pixel. *PureRef*. The reference board's way of working (ADR 0089). 2026. URL: https://www.pureref.com/ (cited on pp. 11, 13, 38, 44, 61).

[75] *Jolt Physics*. Comp. software. Version 5.6.0. Play-mode rigid bodies. Licence: MIT. Cited by licenses/README.md. 2026. URL: https://github.com/jrouwe/JoltPhysics (cited on p. 57).

[76] Krita Foundation. *Krita*. GPL-3.0. The canvas-turning keys (4, 5, 6) the Draw tab answers. 2026. URL: https://krita.org/ (cited on pp. 37, 44, 61).

[77] lilxyzw. *lilToon*. MIT. Its lighting model's formulas (lil_common_frag.hlsl) are re-implemented in src/render/shading.cpp (ADR 0100); no code is copied. 2026. URL: https://github.com/lilxyzw/lilToon (cited on pp. 19, 22, 43, 44).

[78] Microsoft. *ONNX Runtime*. Version 1.31.0. MIT. The optional download that runs the inbetweening network (ADR 0046); only its C API's headers are in the tree. 2026. URL: https://onnxruntime.ai/ (cited on p. 43).

[79] Nekki. *Cascadeur*. The ideas behind physics for animation: the centre of mass, contacts, ballistic flight, AutoPhysics and AutoPosing (ADRs 0042-0045, 0087). Ideas only; no code. 2026. URL: https://cascadeur.com/ (cited on pp. 29, 30, 43, 44).

[80] pixiv Inc. *VRoid Studio*. The VRM avatars and the J_Bip bone names Blendity's imports and the humanoid map read. 2026. URL: https://vroid.com/en/studio (cited on p. 44).

[81] Poiyomi. *Poiyomi Toon Shader*. Read for its material properties (the Unity material converter and the Poiyomi texture preset). 2026. URL: https://github.com/poiyomi/PoiyomiToonShader (cited on pp. 43, 44).

[82] REDSIM. *VRC Light Volumes*. The Light Volume component and its baking (rounds 397-398, ADR 0097); the package stays in the user's Unity project. 2026. URL: https://github.com/REDSIM/VRCLightVolumes (cited on p. 44).

[83] The Khronos Group. *OpenXR*. The VR View's runtime API (ADR 0096). 2026. URL: https://www.khronos.org/openxr/ (cited on p. 39).

[84] VRM Consortium. *UniVRM / MToon*. MIT. The VRM 0.x / 1.0 glTF extension and the MToon material Blendity's VRM import reads. 2026. URL: https://github.com/vrm-c/UniVRM (cited on pp. 22, 43, 44).

[85] Yu Higuchi and the MMD community. *MikuMikuDance (PMX, PMD and VMD file formats)*. The .pmx / .pmd model and .vmd motion files Blendity imports (ADR 0047). 2026. URL: https://sites.google.com/view/vpvp/ (cited on p. 44).

Manuals and websites

[86] Adobe. *Adobe Photoshop User Guide*. The Adobe keymap preset (B, E, V, [ ]), the 27 blend modes, Free Transform and the Layers window Blendity follows. 2026. URL: https://helpx.adobe.com/photoshop/user-guide.html (cited on pp. 11, 38, 44, 61).

[87] Adobe. *Adobe Substance 3D Painter Documentation*. Mesh maps, the layer stack, smart masks and smart materials (ADRs 0079-0082). 2026. URL: https://helpx.adobe.com/substance-3d-painter/home.html (cited on pp. 11, 21, 22, 44, 60, 61).

[88] Autodesk. *Autodesk 3ds Max Documentation*. The 3ds Max keymap preset (Chamfer, Cut, Clone). 2026. URL: https://help.autodesk.com/view/3DSMAX/ (cited on p. 44).

[89] Autodesk. *Autodesk Maya Documentation*. The Maya keymap preset (Q W E R, F8-F11, Set Key on S) and the Maya editor feel. 2026. URL: https://help.autodesk.com/view/MAYAUL/ (cited on p. 44).

[90] Blender Foundation. *Blender 5.2 Reference Manual*. Cited in the user guide's comparisons and the HCI reports. 2026. URL: https://docs.blender.org/manual/en/latest/ (cited on pp. 18, 22, 27, 56, 60, 61).

[91] *Blender Manual: glTF 2.0*. Cited by Learn: Modifiers vs Components. Blender Foundation. 2026. URL: https://dev.epicgames.com (cited on p. 56).

[92] *Blender Manual: Render Output, FFmpeg Video*. Cited by Learn: Modifiers vs Components. Blender Foundation. 2026. URL: https://docs.blender.org (cited on p. 56).

[93] *Blender Manual: Sculpting, Brush Stroke (Use Tilt) and Pressure Curve*. Cited by Learn: Modifiers vs Components. Blender Foundation. 2026. URL: https://docs.blender.org (cited on p. 56).

[94] *Blender Manual: Video Sequencer, Sound Strips*. Cited by Learn: Modifiers vs Components. Blender Foundation. 2026. URL: https://docs.blender.org (cited on p. 56).

[95] *CATS Blender Plugin: Create Visemes*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 56).

[96] *CATS Blender Plugin: Custom Model, Translate and Export Model*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 56).

[97] *CATS Blender Plugin: Decimation, Bone Root and Bone Merging*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 56).

[98] *CATS Blender Plugin: Eye Tracking and Material Combiner*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 56).

[99] *CATS Blender Plugin: Fix Model*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 56).

[100] *CATS Blender Plugin: Model Options and Pose tools*. Cited by Learn: Modifiers vs Components. absolute-quantum (MIT). 2026. URL: https://github.com/absolute-quantum/cats-blender-plugin (cited on p. 56).

[101] Epic Games. *Unreal Engine Documentation*. The Unreal keymap preset and editor feel, FBX collision naming, Sequencer, Live Link. 2026. URL: https://dev.epicgames.com/documentation/ (cited on p. 44).

[102] *Godot Docs: Importing 3D scenes*. Cited by Learn: Modifiers vs Components. Godot Engine contributors. 2026. URL: https://docs.godotengine.org (cited on p. 56).

[103] *Godot Docs: TSCN file format*. Cited by Learn: Modifiers vs Components. Godot Engine contributors. 2026. URL: https://docs.godotengine.org (cited on p. 56).

[104] Godot Engine contributors. *Godot Engine Documentation*. The TSCN format, importing 3D scenes, WorldEnvironment; the Godot bridge. 2026. URL: https://docs.godotengine.org/ (cited on pp. 32, 44).

[105] *lilToon, Poiyomi, MToon and UTS2 documentation*. Cited by Learn: Modifiers vs Components. 2026 (cited on p. 56).

[106] Maxon. *ZBrush Documentation*. Sculpt Mode's brushes, masks, polygroups, DynaMesh and ZRemesher (ADRs 0074-0077). 2026. URL: https://help.maxon.net/zbrush/ (cited on pp. 17, 18, 44, 60).

[107] Pixologic. *Sculptris (dynamic tessellation)*. Dyntopo / Sculptris Pro: the mesh tessellated under the brush (ADR 0078). 2026. URL: https://pixologic.com/sculptris/ (cited on pp. 17, 18, 44, 60).

[108] Plasticity. *Plasticity Documentation*. Construction planes and lines, the perpendicular snap, Shell, Thicken, Draft, Radial Array, Fillet. 2026. URL: https://plasticity.xyz/ (cited on pp. 14, 18, 44).

[109] Sketchbook, Inc. *SketchBook Pro Documentation*. The rulers, ellipse, French curves, symmetry and perspective guides, the toolbar's order, the brush puck (ADRs 0063, 0088). 2026. URL: https://www.sketchbook.com/ (cited on pp. 11, 14, 37, 44, 60, 61).

[110] Trimble. *SketchUp Help Center*. Push/Pull's semantics (ADR 0004), the drawing tools, inference snapping, Follow Me, the SketchUp keymap preset. 2026. URL: https://help.sketchup.com/ (cited on pp. 11, 14, 18, 44).

[111] UModeler, Inc. *UModeler X Documentation*. Drawing shapes onto faces, Follow, Lathe, Slice, the Shift-drag extrude. 2026. URL: https://umodeler.gitbook.io/ (cited on pp. 14, 18, 44).

[112] *Unity Manual: Collider components*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on p. 56).

[113] *Unity Manual: Exporting a package*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on pp. 32, 56).

[114] *Unity Manual: Humanoid Avatars*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com/Manual/AvatarCreationandSetup.html (cited on pp. 29, 30, 56).

[115] *Unity Manual: Importing local asset packages*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on p. 56).

[116] *Unity Manual: Importing LOD meshes*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on p. 56).

[117] *Unity Manual: Lighting > Static GameObjects*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on p. 56).

[118] *Unity Manual: LOD Group*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on p. 56).

[119] *Unity Manual: Prefabs, and the YAML scene format*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on pp. 32, 56).

[120] *Unity Manual: Primitive and placeholder objects*. Cited by Learn: Modifiers vs Components. Unity Technologies. 2026. URL: https://docs.unity3d.com (cited on p. 56).

[121] Unity Technologies. *Unity ProBuilder Package Documentation*. Parametric shapes and the per-mode edit actions the Modeling Tools window compares itself with. 2026. URL: https://docs.unity3d.com/Packages/com.unity.probuilder@latest (cited on pp. 13, 18).

[122] Unity Technologies. *Unity User Manual (Unity 6)*. The editor's windows, shortcuts, Play mode, lighting, the Animation window, Humanoid avatars and the YAML asset formats Blendity follows. 2026. URL: https://docs.unity3d.com/Manual/ (cited on pp. 24, 26, 28, 30, 32, 44, 56, 57, 61).

[123] *Unreal Engine docs: FBX Static Mesh Pipeline, collision (UCX_)*. Cited by Learn: Modifiers vs Components. Epic Games. 2026. URL: https://dev.epicgames.com (cited on p. 56).

[124] *Unreal Engine docs: Sequencer, Camera Cut Track*. Cited by Learn: Modifiers vs Components. Epic Games. 2026. URL: https://dev.epicgames.com (cited on p. 56).

[125] *Unreal Engine: Exporting levels to FBX*. Cited by Learn: Modifiers vs Components. Blender Foundation. 2026. URL: https://dev.epicgames.com (cited on p. 56).

[126] *VRChat Creator Docs: Avatar Descriptor, Eye Look*. Cited by Learn: Modifiers vs Components. VRChat Inc. 2026. URL: https://creators.vrchat.com/avatars/avatar-descriptor (cited on p. 56).

[127] *VRChat Creator Docs: Avatar Descriptor, Lip Sync*. Cited by Learn: Modifiers vs Components. VRChat Inc. 2026. URL: https://creators.vrchat.com/avatars/creating-your-first-avatar (cited on p. 56).

[128] *VRChat Creator Docs: Avatar Performance Ranking System*. Cited by Learn: Modifiers vs Components. VRChat Inc. 2026. URL: https://creators.vrchat.com/avatars/avatar-performance-ranking-system (cited on p. 56).

[129] *VRChat Creators: PhysBones*. Cited by Learn: Modifiers vs Components. VRChat Inc. 2026. URL: https://creators.vrchat.com (cited on p. 56).

[130] VRChat Inc. *VRChat Creator Documentation*. The Avatar Descriptor, PhysBones, the performance ranks, the Worlds SDK; the Avatar window and the Profiler's Budget follow them. 2026. URL: https://creators.vrchat.com/ (cited on pp. 32, 44).

Index C: the Learn lessons and their references

Each lesson of the Learn tab, the Blender source files it points to and the references it cites. Generated by `tools/make_books.py --bib` from `src/editor/lessons.cpp`; FoCG, GEA1 and GEA2 are the three reference books, with the chapter each lesson sends you to.

Learn lesson	Blender source it points to	References
Welcome to Blendity	—	[54, 53, 52]
The Editor Layout	blender/source/blender/editors/screen; blender/source/blender/editors/space_view3d, space_outliner, space_buttons, space_file, space_info; blender/source/blender/editors/space_view3d/view3d_navigate.cc; blender/intern/ghost/intern/GHOST_SystemWin32.cc	[52, 71] (GEA2 16.4 The Game World Editor - why editors are built around a world view, an object tree and a property grid.)
Navigating the Scene View	blender/source/blender/editors/space_view3d/view3d_navigate_view_rotate.cc; blender/source/blender/editors/space_view3d/view3d_navigate_walk.cc	[54, 53, 71] (FoCG 8.1 Viewing Transformations, 8.3 Perspective Projection, 8.5 Field-of-View; GEA1 5.3 Matrices, 5.4 Quaternions - camera orientation is stored as a quaternion here and in Blender)
GameObjects, Components and Objects	blender/source/blender/makesdna/DNA_object_types.h; blender/source/blender/blenkernel/intern/object.cc	[52, 71] (GEA2 16.2 Implementing Dynamic Elements: Game Objects; GEA2 17.2 Runtime Object Model Architectures - object-centric vs property-centric designs)
Hierarchy, Parenting and the Scene Graph	blender/source/blender/editors/object/object_relations.cc; blender/source/blender/editors/space_outliner	[54, 52, 71] (FoCG 12.2 Scene Graphs; GEA2 17.6 Updating Game Objects in Real Time (dependencies between object updates))

Learn lesson	Blender source it points to	References
Move, Rotate, Scale	blender/source/blender/editors/transform; blender/source/blender/editors/gizmo_library; blender/source/blender/editors/object/object_transform.cc	[54, 53, 71] (FoCG 7.2 3D Linear Transformations, 7.3 Translation and Affine Transformations, 7.4 Inverses of Transformation Matrices; GEA1 5.3 Matrices, 5.4 Quaternions, 5.5 Comparison of Rotational Representations)
Coordinate Systems and Handedness	blender/source/blender/io/wavefront_obj	[54, 53, 71] (FoCG 7.5 Coordinate Transformations; GEA1 5.2 Points and Vectors (left- vs right-handed coordinate systems))
Meshes: Vertices, Edges, Faces	blender/source/blender/makesdna/DNA_mesh_types.h; blender/source/blender/blenkernel/intern/mesh_normals.cc	[54, 71] (FoCG 12.1 Triangle Meshes; FoCG 9.2 Operations Before and After Rasterization (interpolating attributes))
Edit Mode: Modeling Like Blender	blender/source/blender/bmesh/operators/bmo_extrude.cc; blender/source/blender/editors/mesh/editmesh_extrude.cc; blender/source/blender/editors/transform/transform_mode_translate.cc	[54, 71] (FoCG 12.1 Triangle Meshes (mesh connectivity))
Modifiers vs Components	blender/source/blender/ikplugin/intern/iksolver_plugin.cc; blender/source/blender/blenkernel/intern/subdiv_mesh.cc; blender/source/blender/modifiers/intern/MOD_smooth.cc	[50, 1, 27, 95, 127, 99, 114, 100, 97, 128, 98, 126, 96, 112, 117, 123, 118, 116, 124, 94, 92, 120, 93, 115, 119, 105, 129, 113, 103, 102, 91, 125, 5, 71]
Cameras and Projection	blender/intern/cycles/kernel/camera/camera.h; blender/intern/cycles/kernel/sample/mapping.h	[54, 52, 71] (FoCG 4.3 Computing Viewing Rays, 13.4 Choosing Random Points (on the aperture), 14.10 Monte Carlo Ray Tracing; FoCG 8.3 Perspective Projection, 8.4 Some Properties of the Perspective Transform, 8.5 Field-of-View; GEA2 11.2 Lights, Camera, Action!)
Overlays: See the Mesh, Hide the Rig	—	[122, 90]

Learn lesson	Blender source it points to	References
How This Editor Renders	blender/source/blender/draw/intern/draw_manager.cc; blender/source/blender/gpu/intern/gpu_select.cc	[54, 52, 53, 71] (FoCG 9.1 Rasterization, 9.2 Operations Before and After Rasterization, 9.4 Culling Primitives for Efficiency; GEA2 11.3 Foundations of 3D Rendering, 11.4 Programming the 3D Graphics Pipeline, 11.5 Pipeline Management; GEA1 4.3 Explicit Parallelism, 4.6 Thread Synchronization Primitives)
Lights and Materials	blender/intern/cycles/kernel/light/spot.h, blender/intern/cycles/kernel/light/area.h; blender/intern/cycles/kernel/svm/blackbody.h	[54, 52, 71] (FoCG ch. 5 Surface Shading; GEA2 12.3 The Rendering Equation, 12.4 The Shading Equation, 12.5 Lighting with Triangle Rasterization)
Play Mode and the Game Loop	—	[53, 52, 122, 75] (GEA1 8.2 The Game Loop, 8.5 Measuring and Dealing with Time; GEA2 14.3 The Collision Detection System, 14.4 Rigid Body Dynamics; GEA2 17.6 Updating Game Objects in Real Time)
Assets, Scenes and Files	blender/source/blender/blenloader/intern/writefile.cc; blender/source/blender/io/wavefront_obj; blender/scripts/addons_core/io_scene_gltf2/blender/imp/blender_gltf.py; blender/source/blender/python/intern/bpy_app_timers.cc	[53, 52, 71] (GEA1 7.1 File System, 7.2 The Resource Manager; GEA2 17.3 World Chunk Data Formats, 17.4 Loading and Streaming Game Worlds)
Undo, Selection and Memory	blender/source/blender/blenlib/BLI_implicit_sharing.hh; blender/source/blender/editors/undo	[53, 71] (GEA1 6.2 Memory Management, 6.3 Containers)
Performance and Stress Testing	—	[53, 54, 71] (GEA1 2.3 Profiling Tools, 10.8 In-Game Profiling, 4.10 SIMD/Vector Processing; FoCG 12.3 Spatial Data Structures)
References	—	[43, 5, 25, 28, 31, 45, 2, 7, 48, 14, 40, 24, 15, 42, 8, 47, 29, 10, 39, 9, 17, 38, 23, 35, 54] (FoCG 11.5 Procedural 3D Textures)

Learn lesson	Blender source it points to	References
UV Mapping	blender/source/blender/geometry/intern/uv_parametrizer.cc; blender/source/blender/geometry/intern/uv_pack.cc; blender/source/blender/editors/uvedit; blender/source/blender/geometry/intern/uv_parametrizer.cc	[25, 54, 49, 41, 71] (FoCG 11.2 Texture Coordinate Functions, 11.1 Looking Up Texture Values)
Materials and Textures	blender/source/blender/nodes/shader/nodes/node_shader_bsdf_principled.cc; blender/intern/cycles/kernel/svm/checker.h, noise.h	[54, 52, 71] (FoCG 11.3 Antialiasing Texture Lookups, 11.4 Applications of Texture Mapping, 11.5 Procedural 3D Textures; FoCG 18.2 Color Spaces; GEA2 11.3 Foundations of 3D Rendering (textures, materials))
Render Engines	blender/source/blender/draw/engines/eevee; blender/intern/cycles/kernel/integrator; blender/intern/cycles/bvh	[52, 54, 71] (GEA2 11.4 Programming the 3D Graphics Pipeline, 12.5 Lighting with Triangle Rasterization; FoCG 4 Ray Tracing, 12.3 Spatial Data Structures, 13.3 Monte Carlo Integration, 14.10 Monte Carlo Ray Tracing; GEA2 12.3 The Rendering Equation, 12.6 Lighting with Stochastic Ray Tracing)
Light, Sky and Tone Mapping	blender/intern/sky/source/sky_hosek.cpp; blender/source/blender/imbuf/intern/colormanagement.cc	[54, 52, 17, 71] (FoCG 20 Tone Reproduction (20.2 Dynamic Range, 20.9 Sigmoids); FoCG 5.3 Ambient Illumination; GEA2 12.2 Radiometry and the Theory of Light Transport, 12.7 Post-Processing)
Editing: Develop a Render	—	[57, 56]
Baked and Realtime Lighting	blender/source/blender/draw/engines/eevee	[45, 52, 71] (GEA2 12.6 Lighting with Stochastic Ray Tracing)
Advanced Modeling Tools	blender/source/blender/bmesh/intern/bmesh_walkers_impl.cc; blender/source/blender/editors/mesh/editmesh_loopcut.cc; blender/source/blender/modifiers/intern/MOD_mirror.cc, MOD_array.cc, MOD_solidify_extrude.cc	[54, 71] (FoCG 12.1 Triangle Meshes (mesh connectivity), 12.2 Scene Graphs)

Learn lesson	Blender source it points to	References
Libraries and Performance	blender/build_files/build_environment/cmake; blender/intern/cycles/bvh/embree.cpp, intern/cycles/integrator/denoiser_oidn.cpp, intern/cycles/integrator/guiding.h; blender/source/blender/geometry/intern/mesh_boolean_manifold.cc; blender/intern/cycles/util/simd.h, blender/source/blender/blenlib/intern/math_color.cc; blender/source/blender/blenlib/BLI_task.hh	[52, 53, 54, 71] (GEA2 14.2 Collision/Physics Middleware, 14.5 Integrating a Physics Engine into Your Game; GEA1 2.3 Profiling Tools; FoCG 22.3 Optimization Techniques; FoCG 9.4 Culling Primitives for Efficiency, 12.3 Spatial Data Structures; GEA1 4.10 SIMD/Vector Processing, 3.5 Memory Architectures, 3.3 Data, Code and Memory Layout; GEA1 4.3 Explicit Parallelism, 4.8 Some Rules of Thumb for Concurrency)
Bevel, Bridge and Push Through	blender/source/blender/bmesh/tools/bmesh_bevel.cc, blender/source/blender/editors/mesh/editmesh_bevel.cc; blender/source/blender/bmesh/operators/bmo_bridge.cc; blender/source/blender/windowmanager/intern/wm_operators.cc; blender/source/blender/editors/undo/ed_undo.cc	[54, 53, 71] (FoCG 15.5 Cubics (the Hermite curve the tube follows), 12.1 Triangle Meshes; FoCG 21.5 Constructive Solid Geometry; GEA1 1.6 Tools and the Asset Pipeline)
Rendering on the GPU	blender/intern/cycles/device; blender/intern/cycles/kernel/integrator; blender/intern/cycles/session/session.cpp; blender/intern/cycles/integrator/path_trace.cpp, blender/intern/cycles/integrator/work_balancer.cpp; blender/intern/cycles/integrator/denoiser_oidn.cpp, blender/intern/cycles/integrator/denoiser_oidn_gpu.cpp; blender/source/blender/gpu/opengl/gl_debug.cc; blender/source/blender/editors/sculpt_paint/mesh/sculpt.cc, blender/source/blender/draw/engines/eevee/eevee_instance.cc	[54, 71] (FoCG 12.3 Spatial Data Structures)

Learn lesson	Blender source it points to	References
Points, Lines, N-gons and Booleans	blender/source/blender/bmesh/operators/bmo_extrude.cc; blender/source/blender/editors/mesh/editmesh_knife.cc; blender/source/blender/bmesh/operators/bmo_dissolve.cc; blender/source/blender/geometry/intern/mesh_boolean_manifold.cc; blender/source/blender/editors/mesh/editmesh_bisect.cc	[71]
Drawing in Perspective	—	[19, 6, 13, 109]
Customizing the Editor	source/blender/windowmanager	[53, 106, 90, 71] (GEA1 ch. 10 Tools for Debugging and Development - the tools pipeline; the editor is the first tool)
Sculpting	source/blender/editors/sculpt_paint	[54, 52, 106, 107, 20, 12, 18, 71] (FoCG ch. 12 Data Structures for Graphics - the half-edge and the triangle mesh the brushes walk; GEA2 ch. 10 The Rendering Engine - why a game wants the retopologized quads, not the sculpt)
Painting the Surface	source/blender/editors/sculpt_paint/paint_image_proj.cc	[54, 52, 87, 71] (FoCG ch. 11 Texture Mapping - UV space, texel footprints and why a seam needs dilation; GEA2 ch. 10.6 Materials, textures and shaders in the rendering pipeline)
The Hand-painted Look	source/blender/editors/sculpt_paint	[54, 52, 3, 71] (FoCG ch. 5 Linear Algebra and ch. 18 Color - value, hue and why the light can be premultiplied into the texture; GEA2 ch. 10.1 Foundations of Depth-Buffered Triangle Rasterization - the unlit path)

Learn lesson	Blender source it points to	References
Stylized Environments	source/blender/editors/gpencil	[54, 52, 71] (FoCG ch. 15 Curves - the ribbon's strip along a path; ch. 11 Texture Mapping - the alpha-cut card; GEA2 ch. 10.4 Visual Effects and Overlays - particle and card systems, fog as a post effect)
Drawing like SketchBook	—	[109, 76, 86]
Papers	—	[109, 74]
Blender's Shader Nodes in the Textures Window	—	[35, 90]
Fog with a Volume Material	—	[90]
Working Faster with Textures and Materials	—	[90, 87]
Flatten to the Camera (the 2D Look)	—	[30]
Shapes by the Camera's Angle	—	[30]
Animating Any Property	—	[122, 90]
The Books and the References	—	[53] (GEA1 1.6 Tools and the Asset Pipeline - why an editor documents its pipeline and its sources)

Colophon

Authorship and copyright. Blendity is copyright Emiliano Manalo and contributors. Its source code, tests, documentation and these two books were written with Anthropic's Claude through Claude Code [70], under a human's direction and review: the user decides what Blendity is for, approves each task's specification and decision record before code is written, reviews every pull request and merges it. The model's outputs belong to the user who directed them under Anthropic's terms [58], and are licensed to the extent copyright subsists in them (the human selection, direction, review and editing); the `Co-Authored-By` trailer on every commit is attribution, not a licence grant. No third-party code is claimed as the project's own: what is copied is listed in `licenses/README.md` with its licence, what is followed is credited in `CREDITS.md` and the References chapter, and the whole story is in `docs/AI_DISCLOSURE.md`.

Licence. The source is GPL-2.0-or-later [60], the same as Blender's [71]; the executables, which combine it with Apache-2.0 libraries [66], are distributed under GPL-3.0-or-later [61]. The scenes, meshes, textures, renders and animations made with Blendity are their maker's: the GPL covers the program, not its output.

How the books are made. The sources are LaTeX under `docs/book/`, typeset with LuaLaTeX in the DejaVu faces (the editor's own) and biblatex with biber for the reference list. `tools/make_books.py` generates the reference list and the index tables from the project's own files, takes the figures headless from a scratch project, builds both PDFs and checks that nothing has drifted. The release workflow builds them again on every tag and attaches them to the release.

Accessibility and readability. The books follow the guidelines the editor itself is measured against (ADR 0085), applied to documents: Nielsen's tenth heuristic, help and documentation [32, 55] (easy to search: a table of contents, bookmarks, a list of figures, a reference list with back-references and three indexes; focused on the user's task: the user guide is the work stage by stage with numbered steps and worked examples, the architecture book the questions a maintainer asks; not too large: each chapter ends in a pointer to the Learn lesson that goes deeper); Carroll's minimalist instruction [51] (start on a real task, keep the reading short, support error recovery: a troubleshooting chapter and the "when something refuses" list); and WCAG 2.1 [46] as PDF/UA applies it [63]: every text colour, link and box title reads at 4.5:1 or more (the ratios are in `blendity-common.sty`), links are told apart by form as well as colour (bracketed citation numbers, "chapter 3", monospace URLs), every figure carries alt text and a caption that says what it shows, the diagrams' meaning is in their captions and the text around them, the document's language and metadata are declared and its headings are the PDF's bookmarks (structure tags for a screen reader are the one open item: LaTeX's tagging project does not yet support the packages the books use, so the captions carry what the alt text would), the body type is about 10 points of a face made for screens with 80 characters a line, and nothing is conveyed by colour alone. Keys, menus and controls are typeset consistently: Ctrl+K is a key, Window > Layouts a menu path, Generate Lighting a control's label, `make_books.py` a file or command.

Version. Blendity 0.13.0, November 2026, from the `main` branch of `https://github.com/EmilianoManaloIV/blendity`.